

An abstract graphic on the left side of the slide, consisting of a complex network of blue lines and dots, resembling a neural network or a data structure, set against a black background.

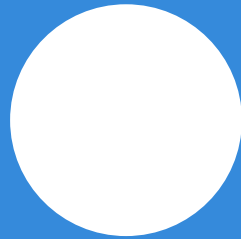
Deep Sequence Modeling

MIT 6.S191

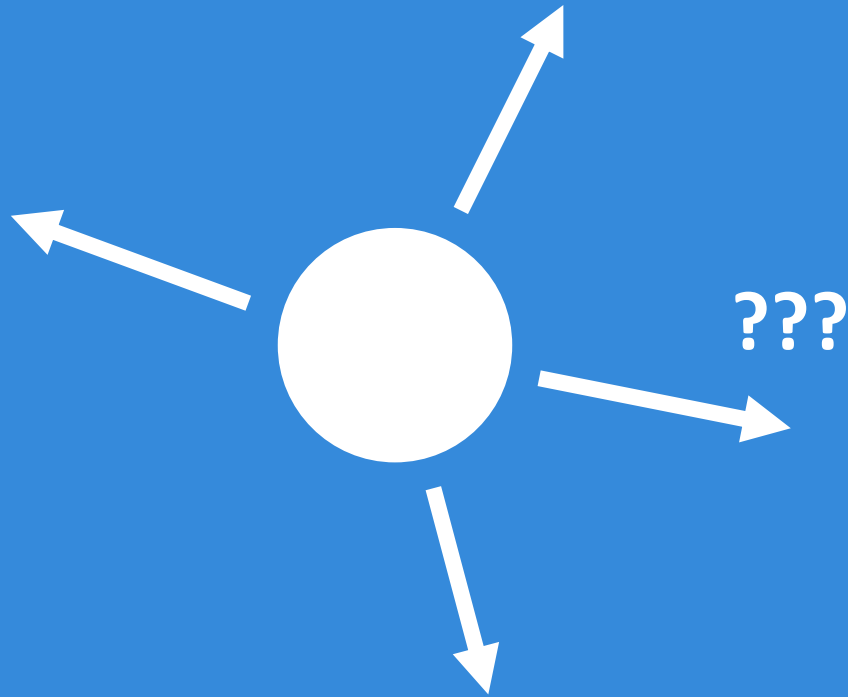
Ava Soleimany
January 28, 2019



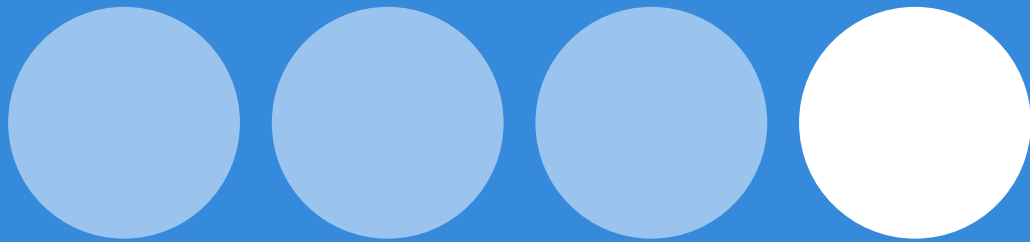
Given an image of a ball,
can you predict where it will go next?



Given an image of a ball,
can you predict where it will go next?



Given an image of a ball,
can you predict where it will go next?



Given an image of a ball,
can you predict where it will go next?



Sequences in the wild



Audio

Sequences in the wild

character:

6.S191 Introduction to Deep Learning

word:

Text

A Sequence Modeling Problem: Predict the Next Word

A sequence modeling problem: predict the next word

“This morning I took my cat for a walk.”

Adapted from H. Suresh, 6.S191 2018

A sequence modeling problem: predict the next word

“This morning I took my cat for a walk.”

given these words

Adapted from H. Suresh, 6.S191 2018

A sequence modeling problem: predict the next word

“This morning I took my cat for a walk.”

given these words

predict the
next word

Adapted from H. Suresh, 6.S191 2018

Idea #1: use a fixed window

“This morning I took my cat for a walk.”

given these
two words

predict the
next word

Adapted from H. Suresh, 6.S191 2018

Idea #1: use a fixed window

“This morning I took my cat for a walk.”

given these predict the
two words next word

One-hot feature encoding: tells us what each word is

[1 0 0 0 0 0 1 0 0 0]

for

a

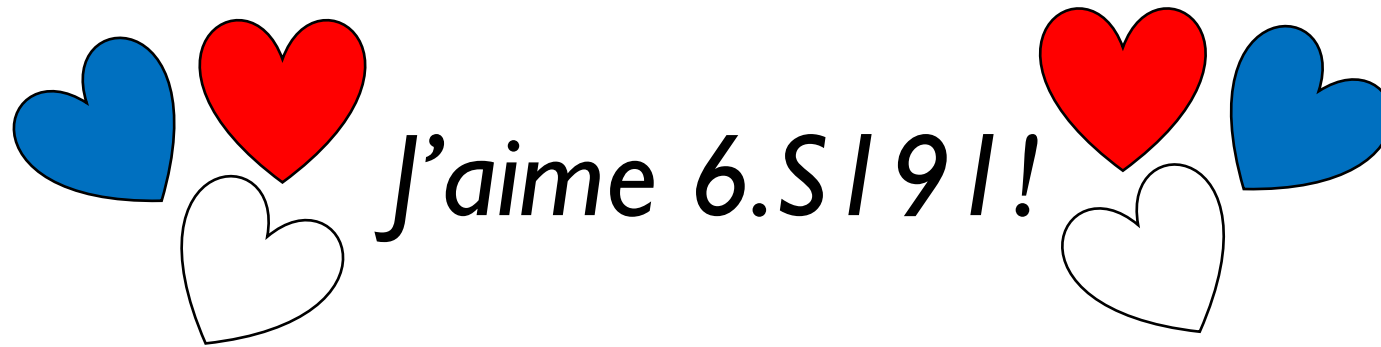


prediction

Adapted from H. Suresh, 6.S191 2018

Problem #1: can't model long-term dependencies

“**France** is where I grew up, but I now live in Boston. I speak fluent ____.”



We need information from **the distant past** to accurately predict the correct word.

Adapted from H. Suresh, 6.S191 2018

Idea #2: use entire sequence as set of counts

“This morning I took my cat for a”



“bag of words”

[0 1 0 0 1 0 0 ... 0 0 1 1 0 0 0 1]



prediction

Adapted from H. Suresh, 6.S191 2018

Problem #2: counts don't preserve order



The food was good, not bad at all.

vs.

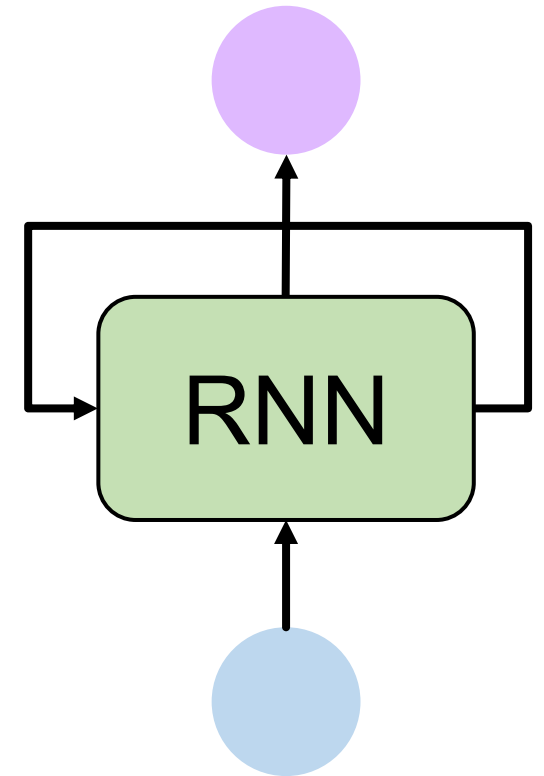
The food was bad, not good at all.



Sequence modeling: design criteria

To model sequences, we need to:

1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence

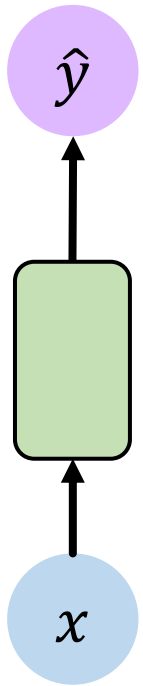


Today: **Recurrent Neural Networks (RNNs)** as
an approach to sequence modeling problems

Adapted from H. Suresh, 6.S191 2018

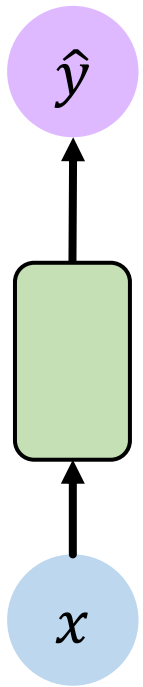
Recurrent Neural Networks (RNNs)

Standard feed-forward neural network

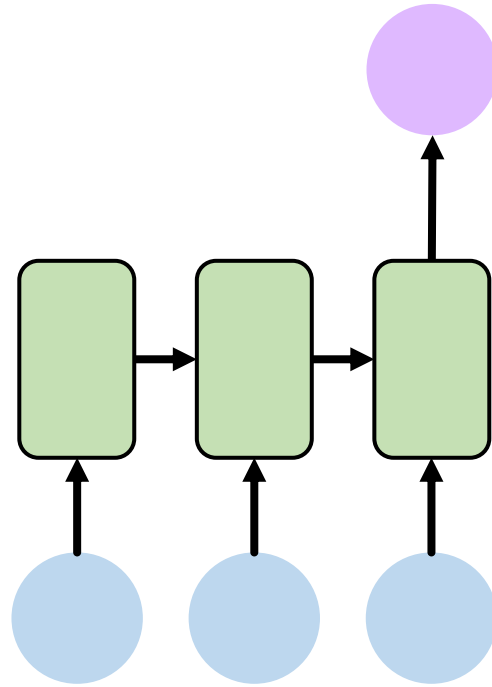


One to One
“Vanilla” neural network

Recurrent neural networks: sequence modeling

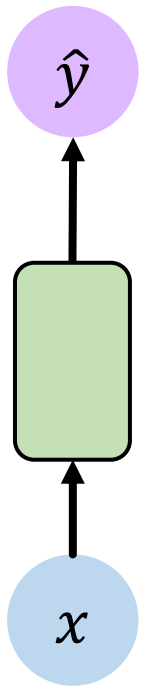


One to One
"Vanilla" neural network

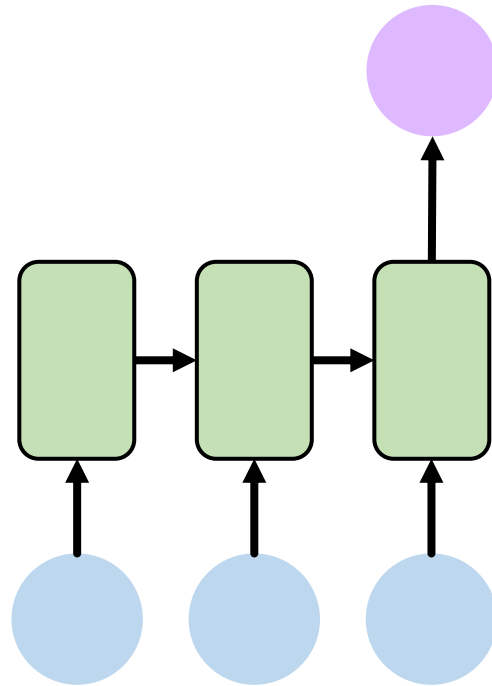


Many to One
Sentiment Classification

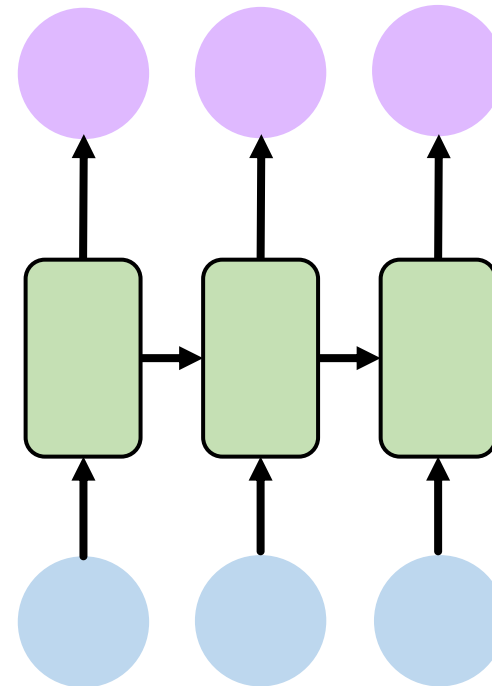
Recurrent neural networks: sequence modeling



One to One
“Vanilla” neural network



Many to One
Sentiment Classification

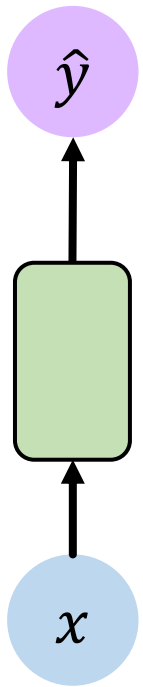


Many to Many
Music Generation

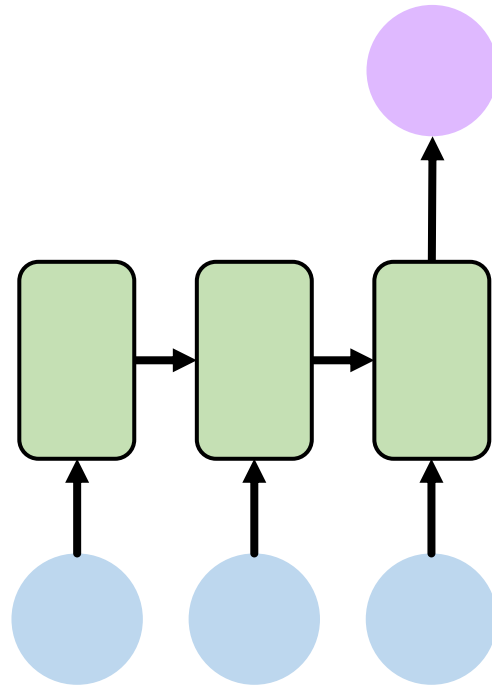


6.S191 Lab!

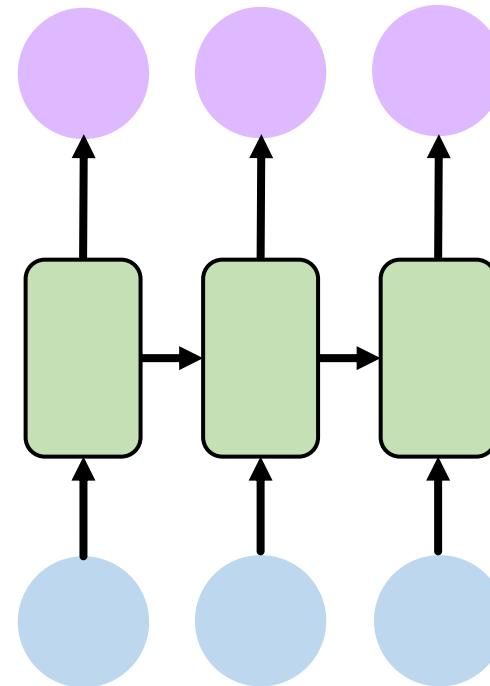
Recurrent neural networks: sequence modeling



One to One
“Vanilla” neural network



Many to One
Sentiment Classification



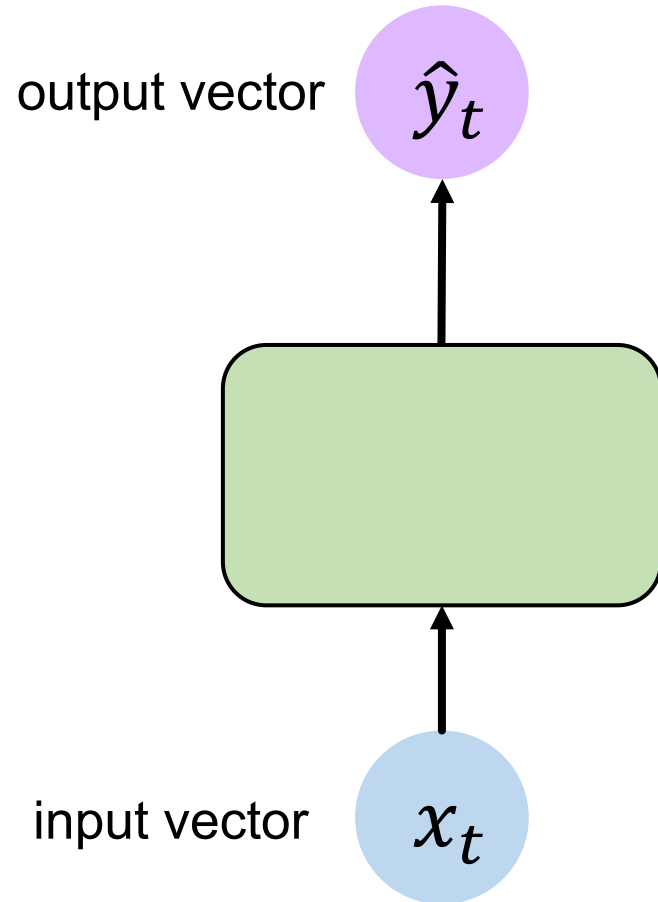
Many to Many
Music Generation

... and many other
architectures and
applications

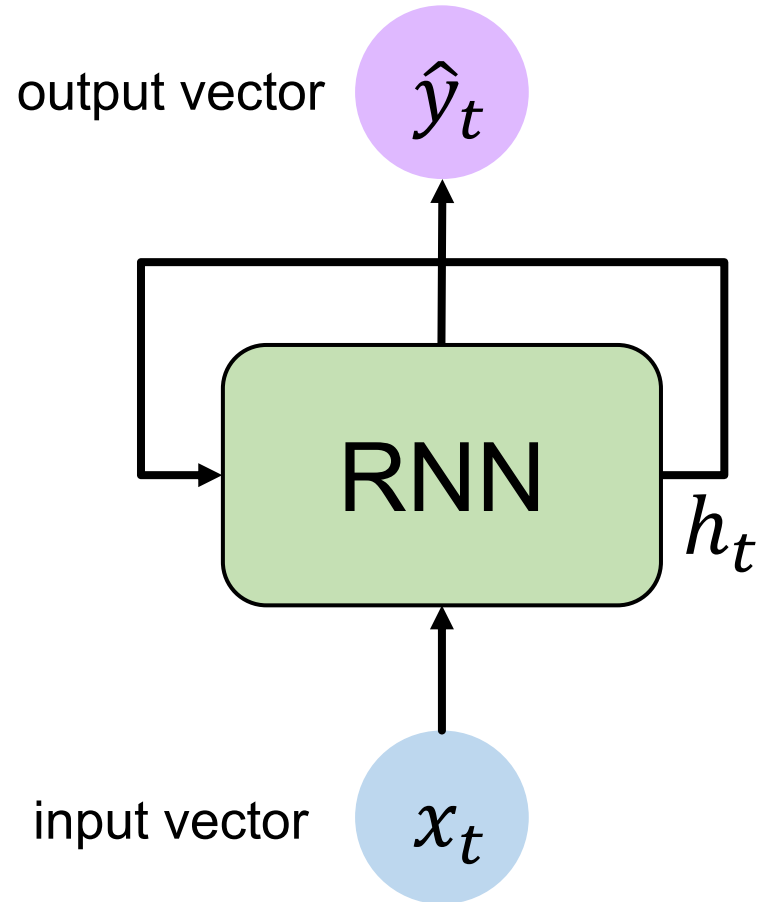


6.S191 Lab!

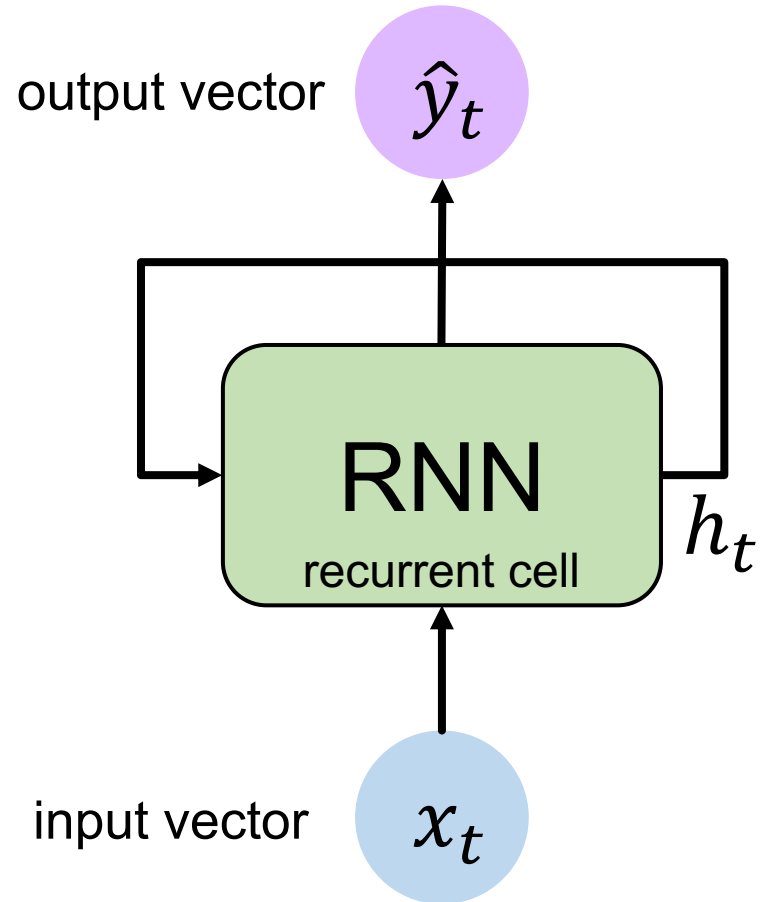
A standard “vanilla” neural network



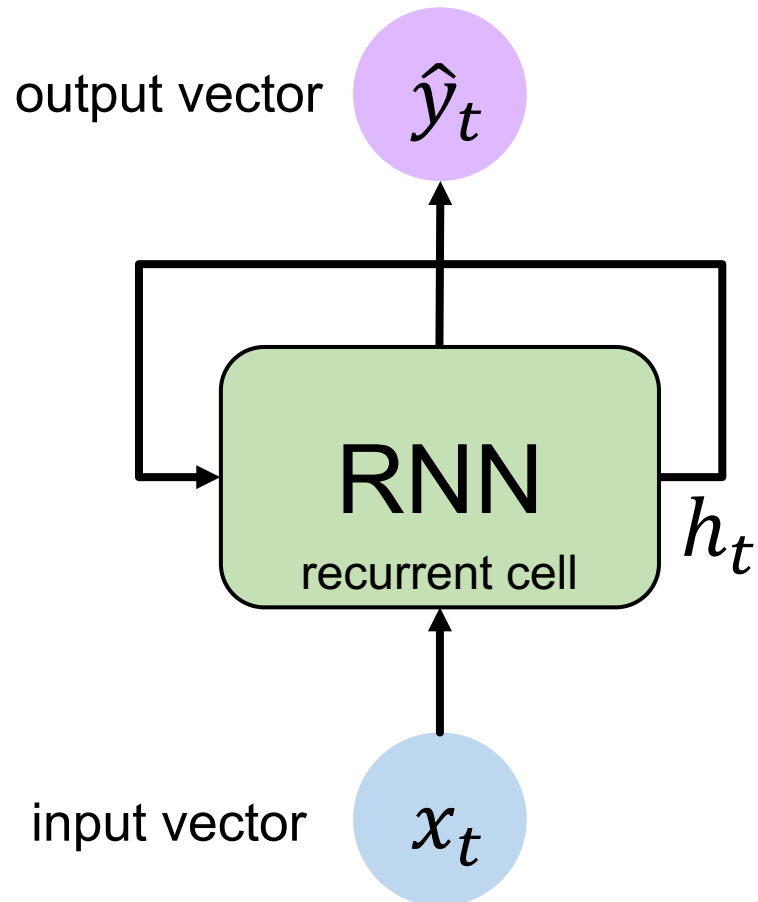
A recurrent neural network (RNN)



A recurrent neural network (RNN)

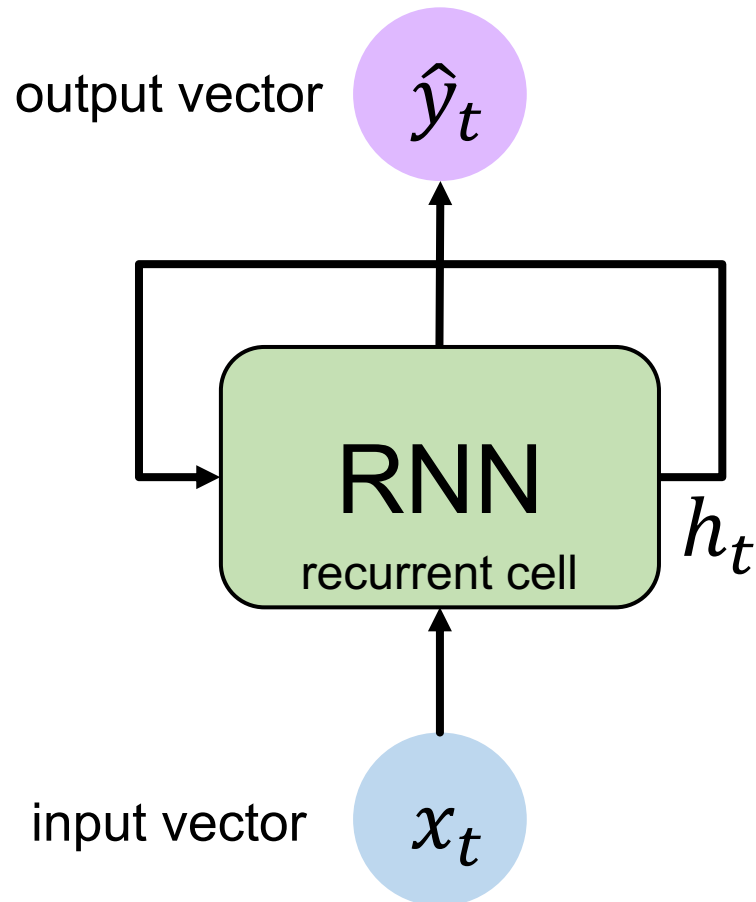


A recurrent neural network (RNN)



Apply a **recurrence relation** at every time step to process a sequence:

A recurrent neural network (RNN)

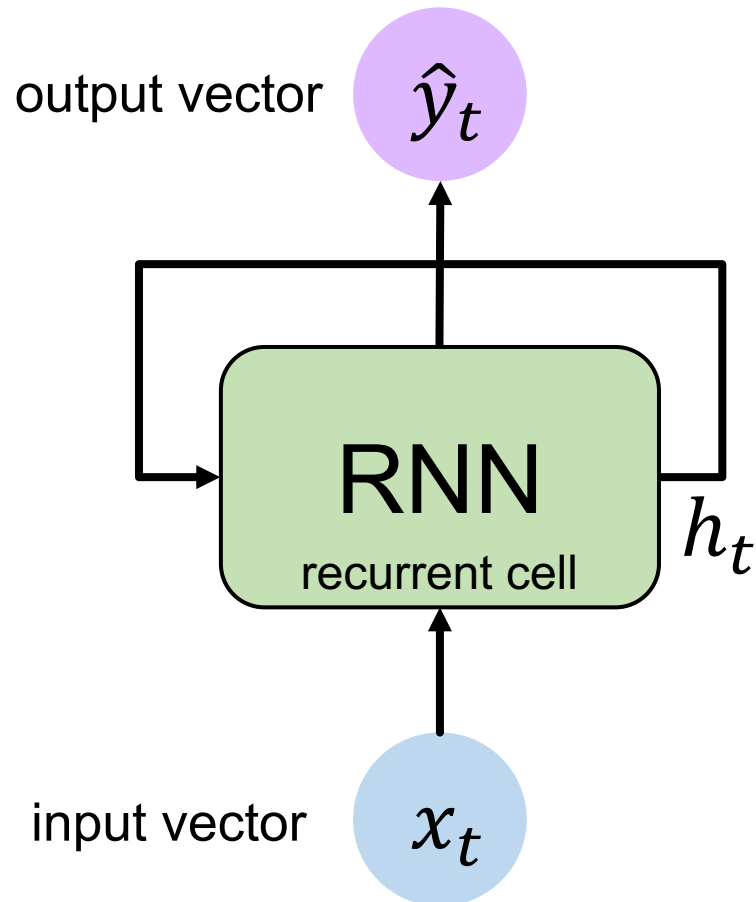


Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

cell state function old state input vector at
parameterized
by W time step t

A recurrent neural network (RNN)



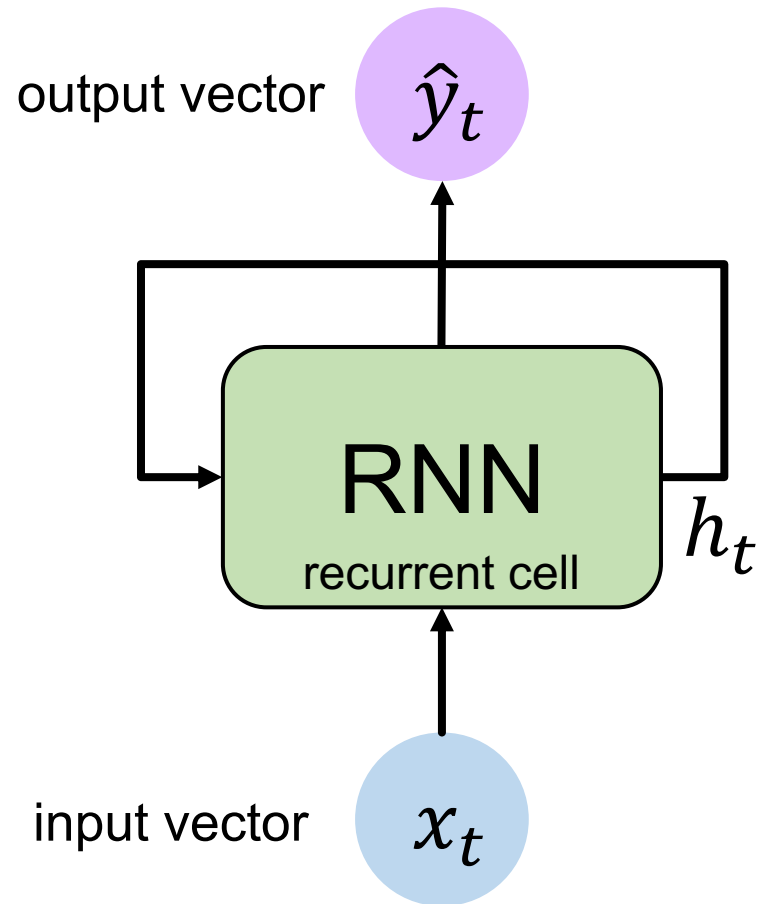
Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

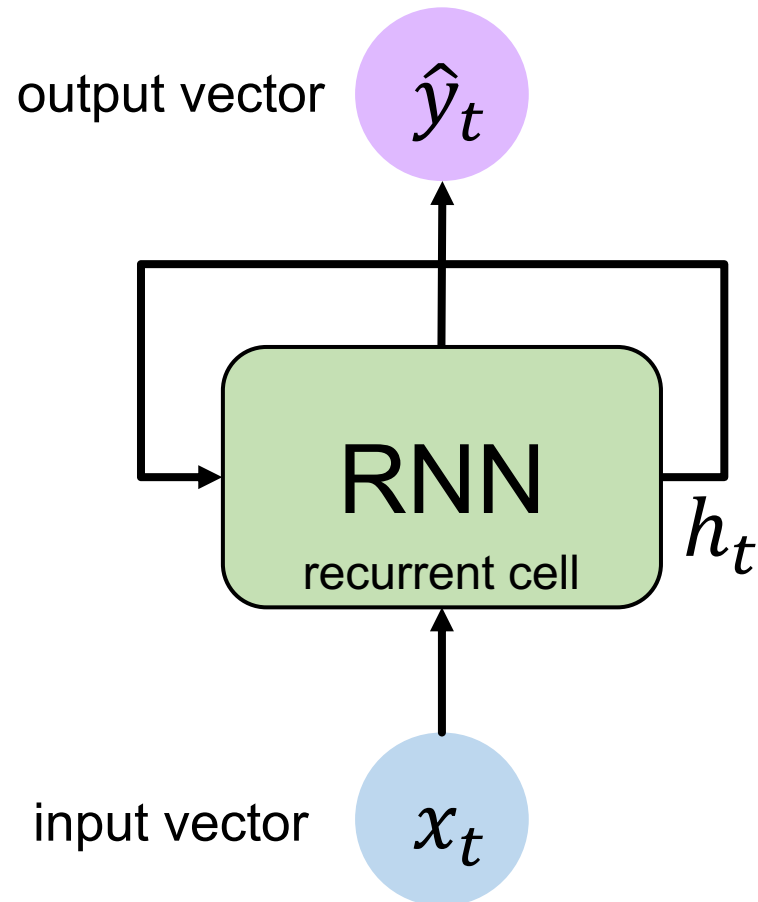
new state function parameterized by W old state input vector at time step t

Note: the same function and set of parameters are used at every time step

RNN state update and output

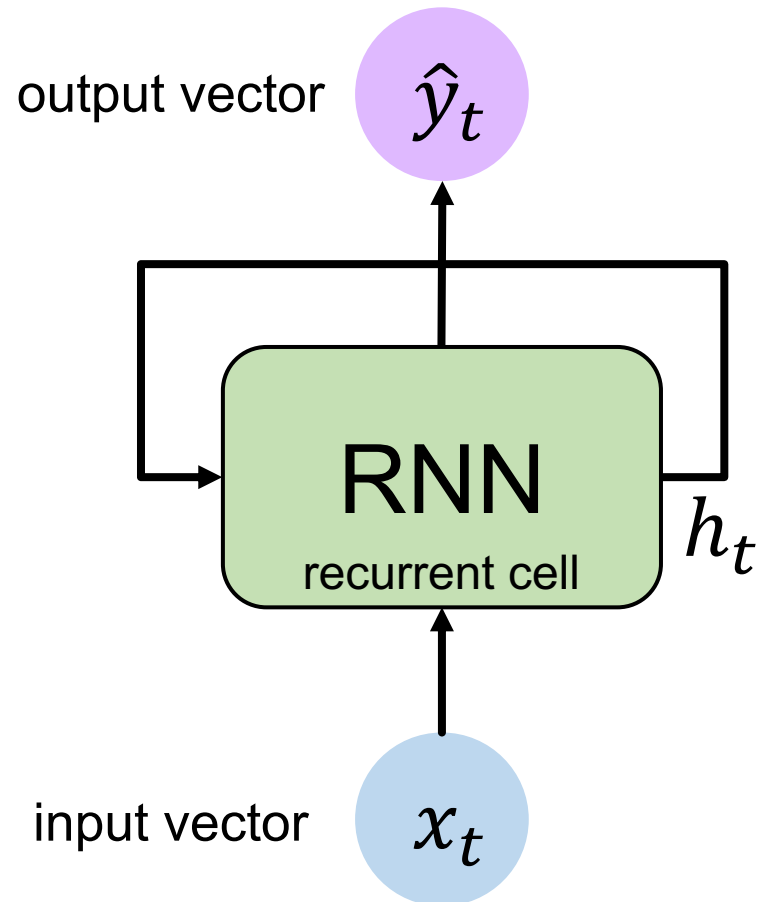


RNN state update and output



Input Vector

RNN state update and output

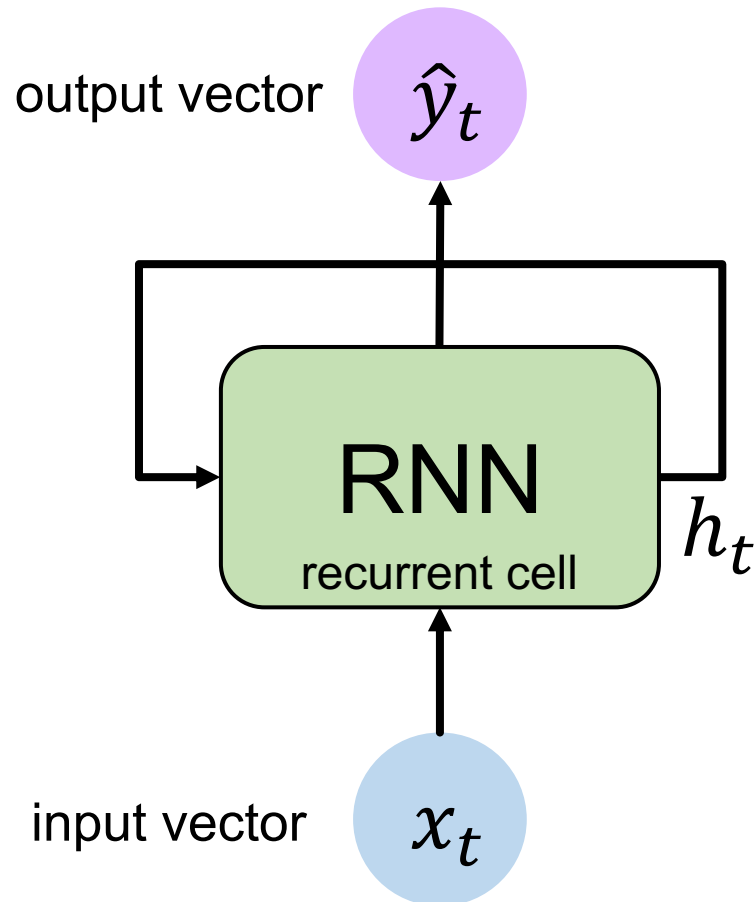


Update Hidden State

$$h_t = \tanh(\mathbf{W}_{hh}h_{t-1} + \mathbf{W}_{xh}x_t)$$

Input Vector

RNN state update and output



Output Vector

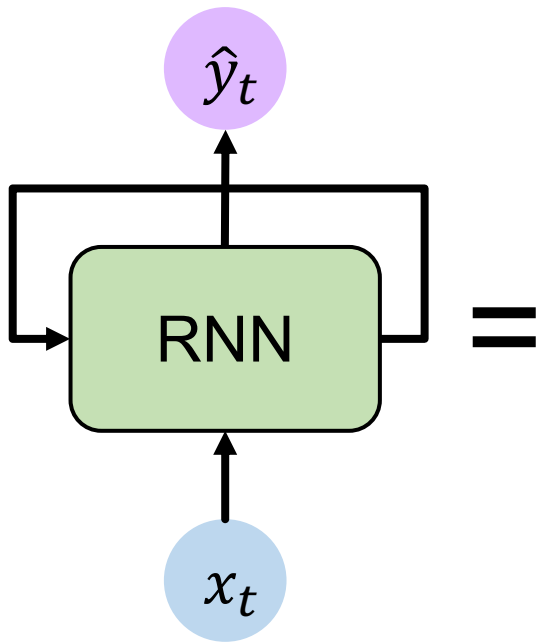
$$\hat{y}_t = \mathbf{W}_{hy}h_t$$

Update Hidden State

$$h_t = \tanh(\mathbf{W}_{hh}h_{t-1} + \mathbf{W}_{xh}x_t)$$

Input Vector

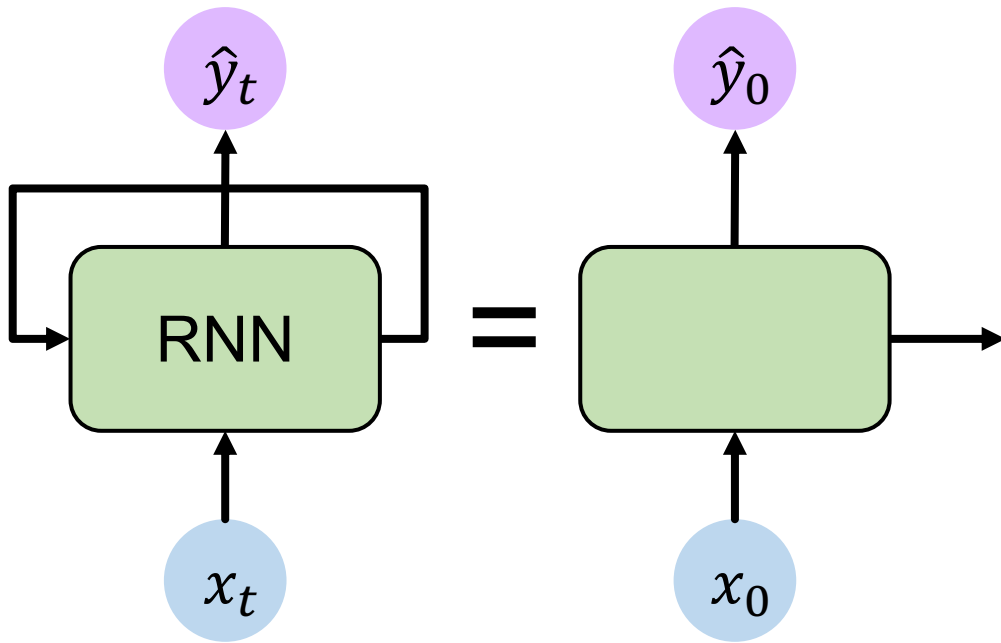
RNNs: computational graph across time



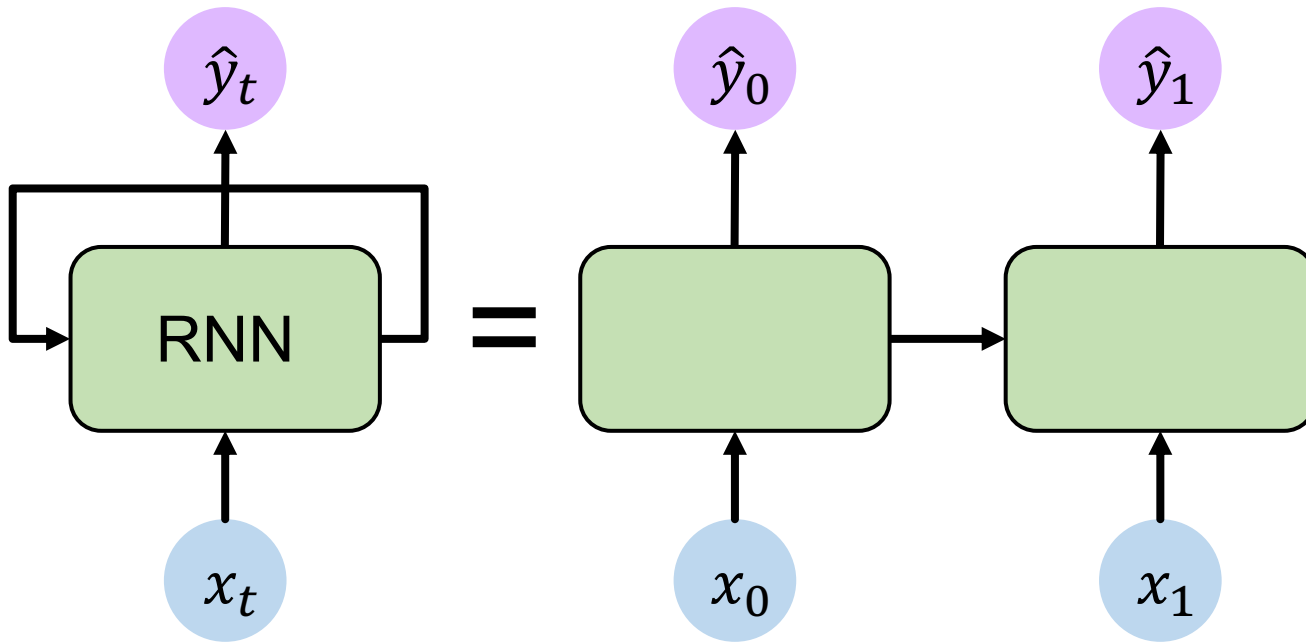
=

Represent as computational graph unrolled across time

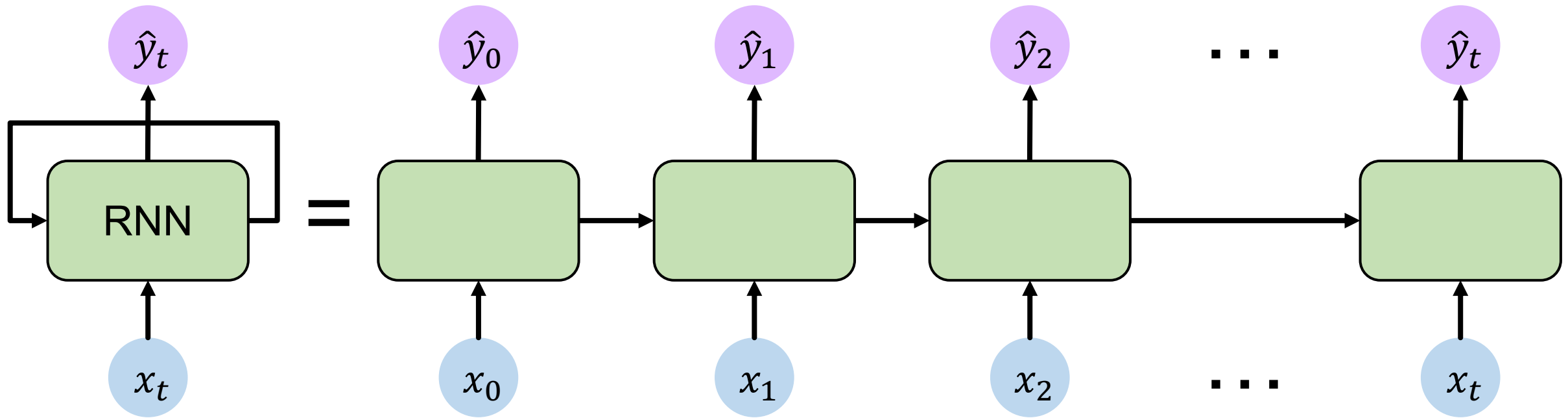
RNNs: computational graph across time



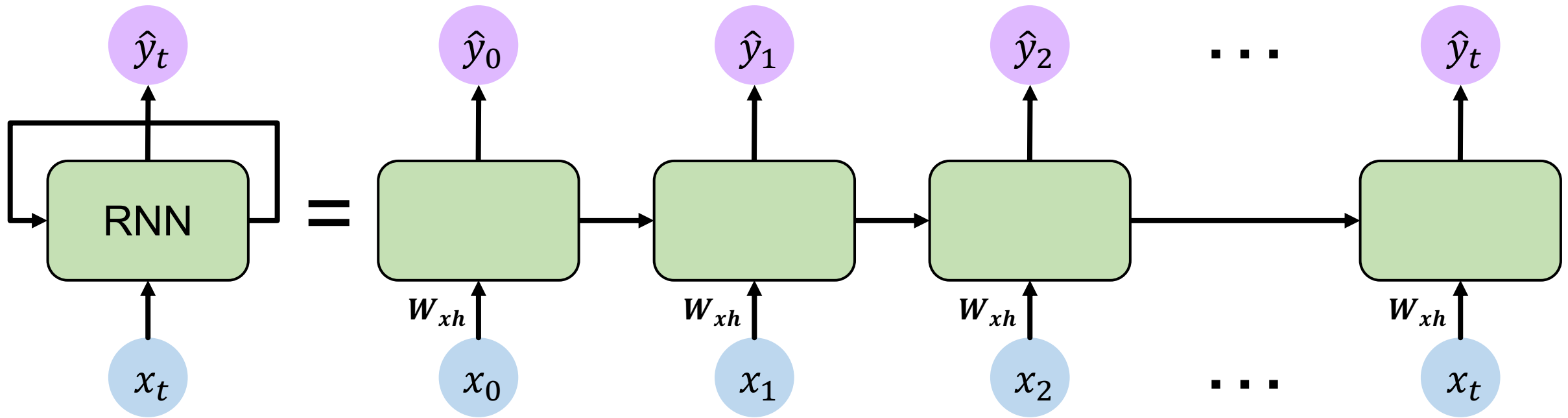
RNNs: computational graph across time



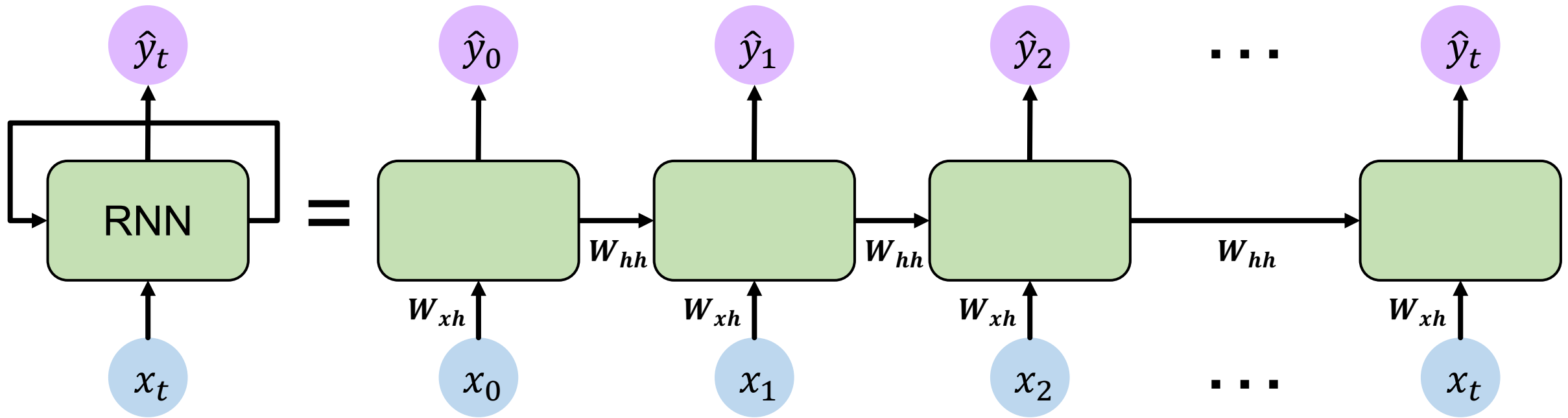
RNNs: computational graph across time



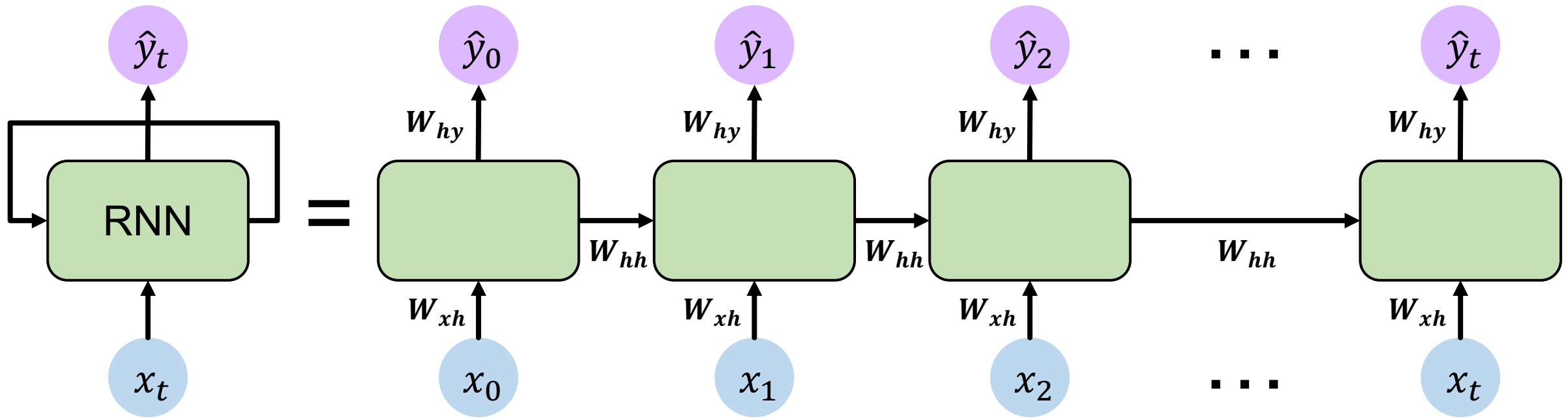
RNNs: computational graph across time



RNNs: computational graph across time

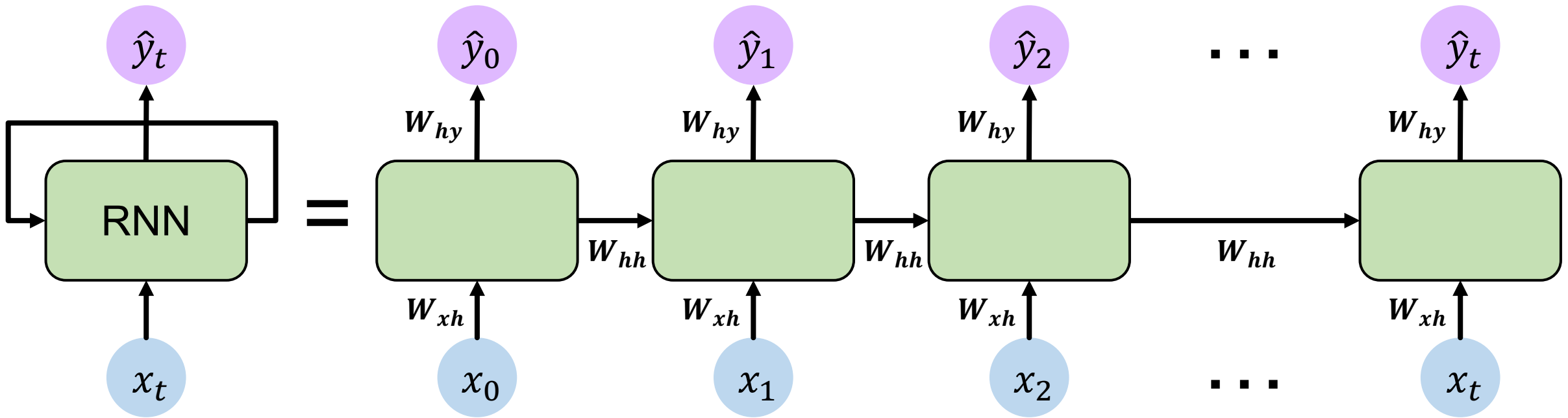


RNNs: computational graph across time



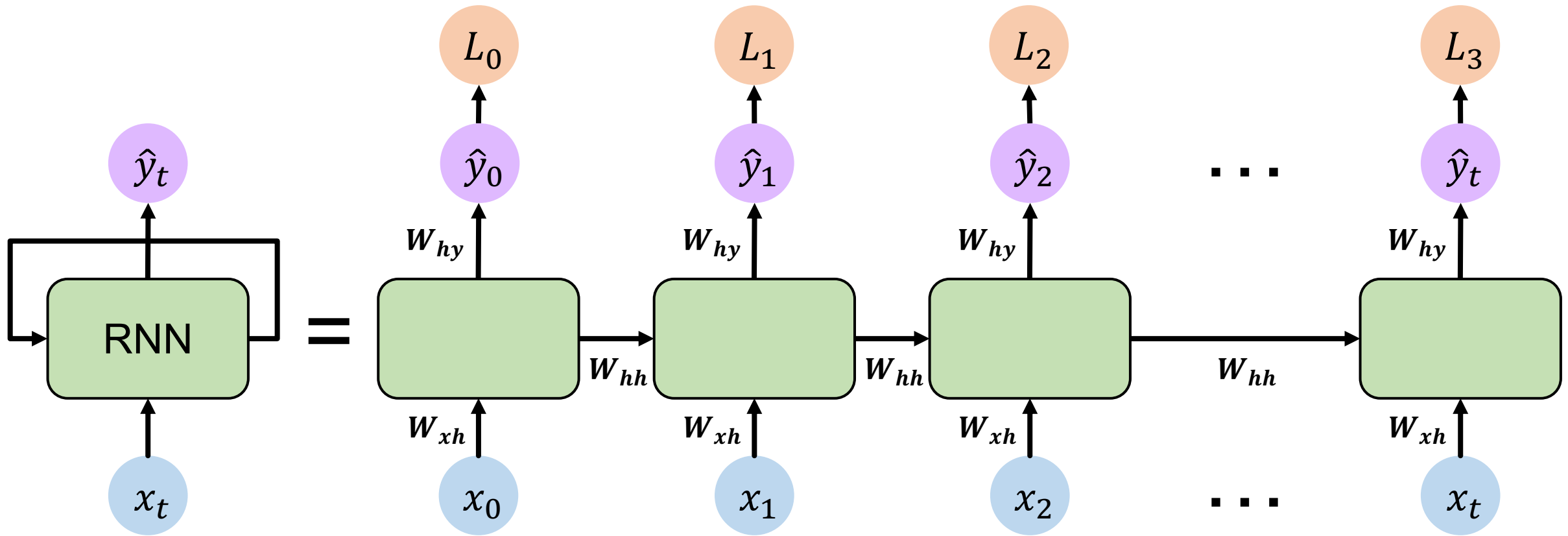
RNNs: computational graph across time

Re-use the **same weight matrices** at every time step

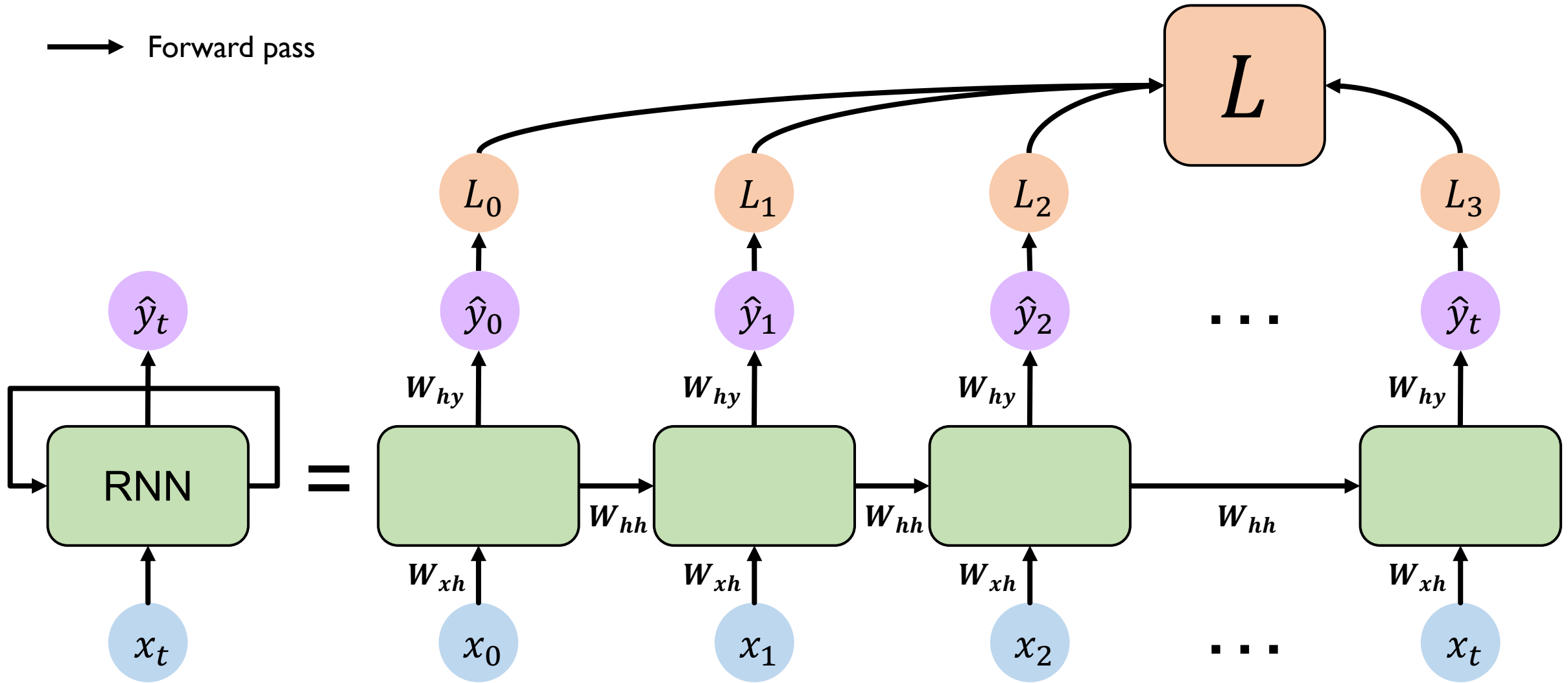


RNNs: computational graph across time

→ Forward pass

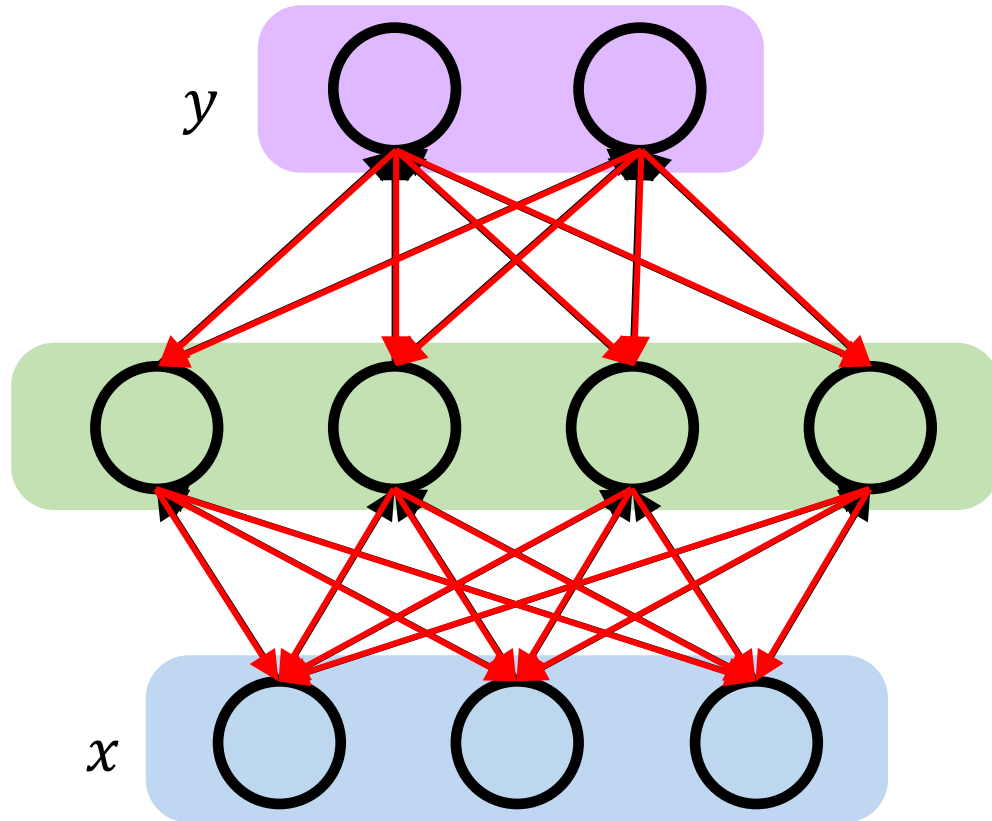


RNNs: computational graph across time



Backpropagation Through Time (BPTT)

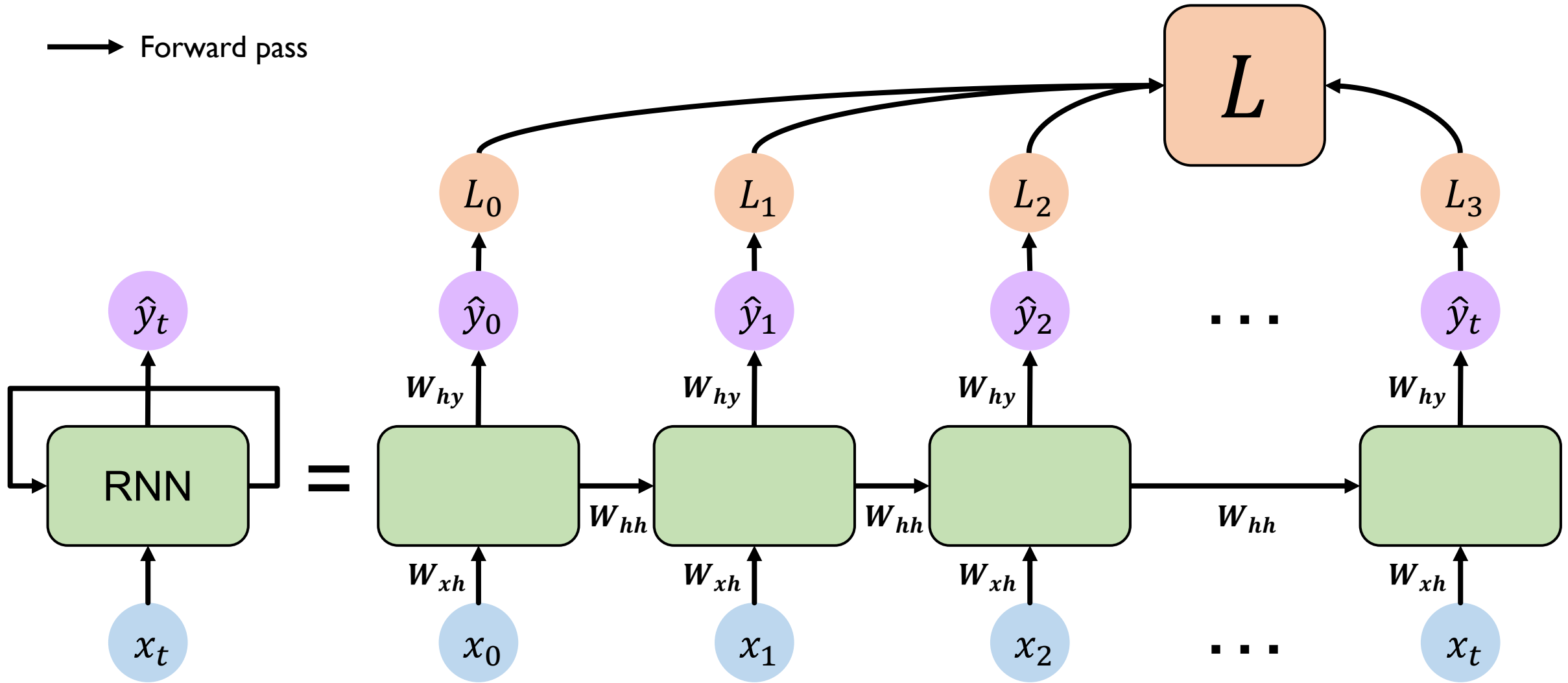
Recall: backpropagation in feed forward models



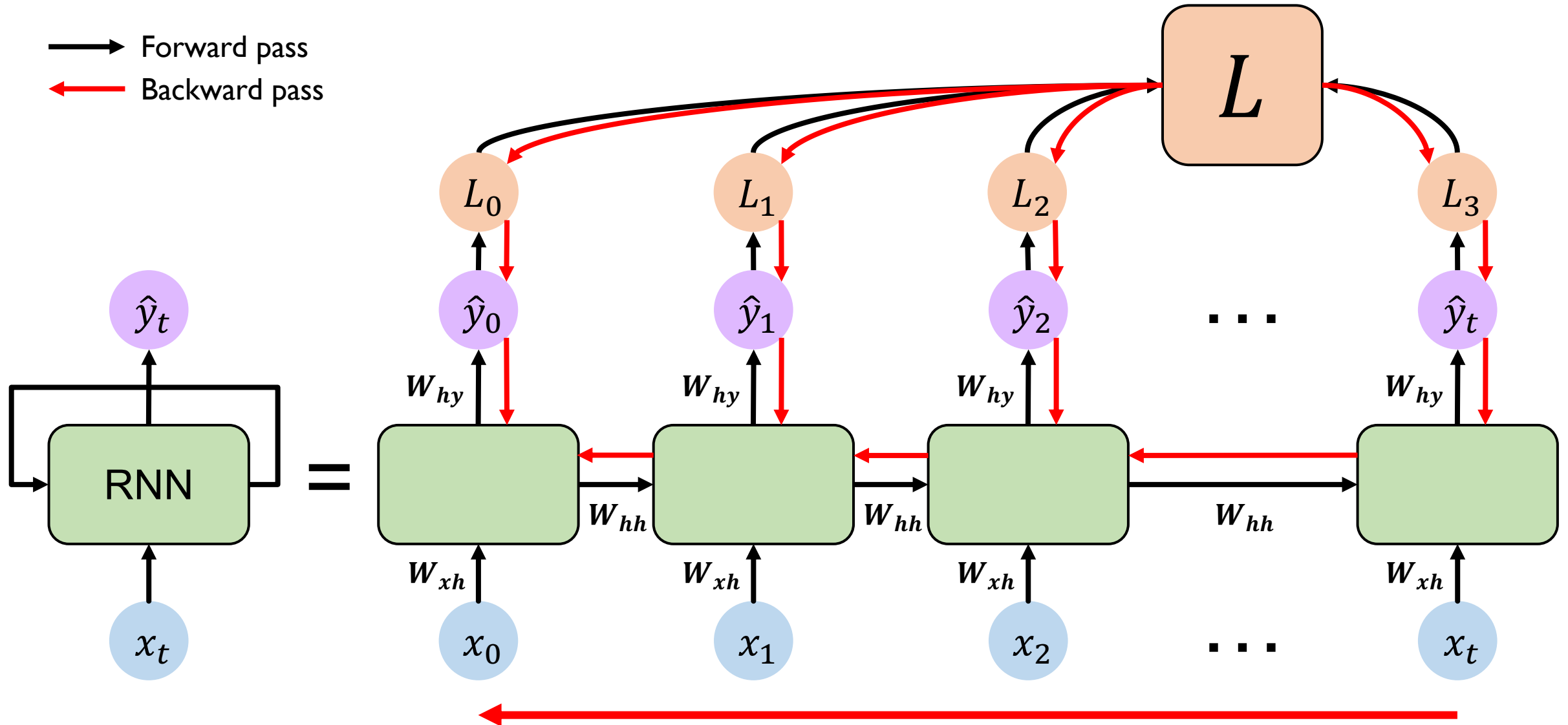
Backpropagation algorithm:

1. Take the derivative (gradient) of the loss with respect to each parameter
2. Shift parameters in order to minimize loss

RNNs: backpropagation through time

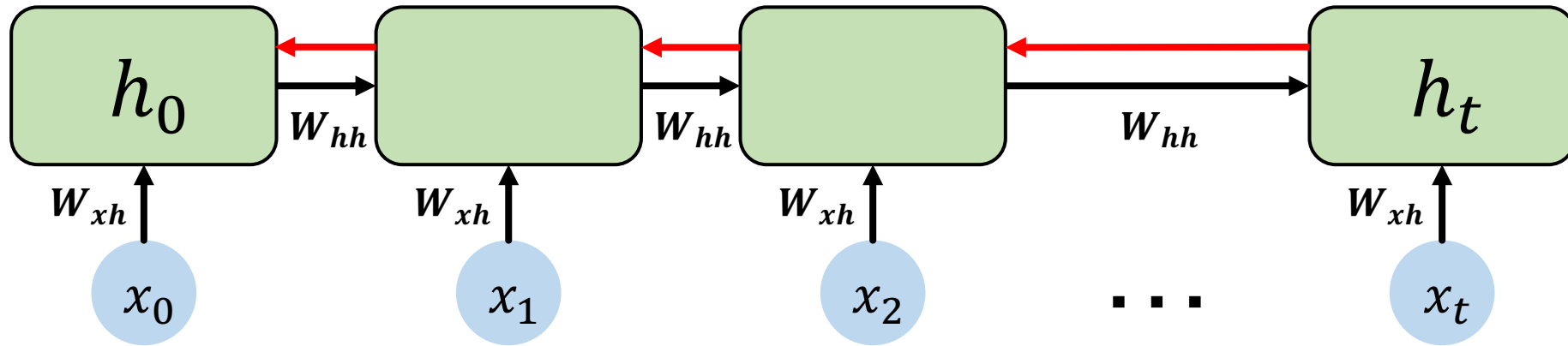


RNNs: backpropagation through time

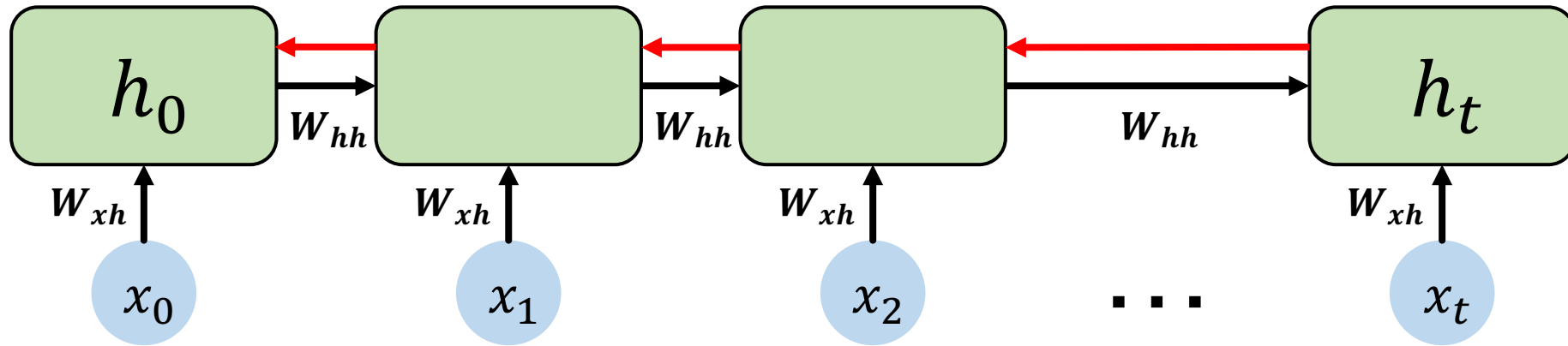


[4]

Standard RNN gradient flow

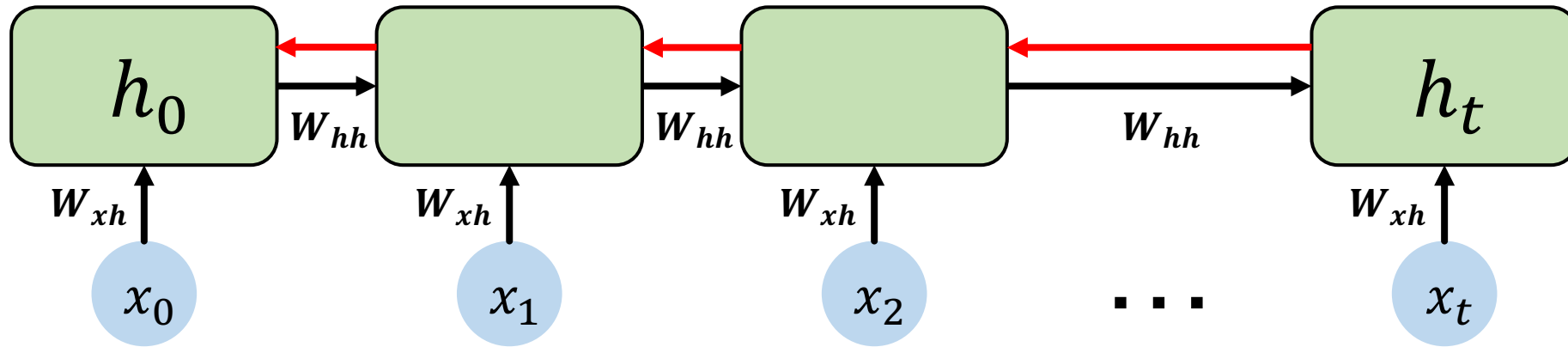


Standard RNN gradient flow



Computing the gradient wrt h_0 involves **many factors of W_{hh}** (and repeated f' !)

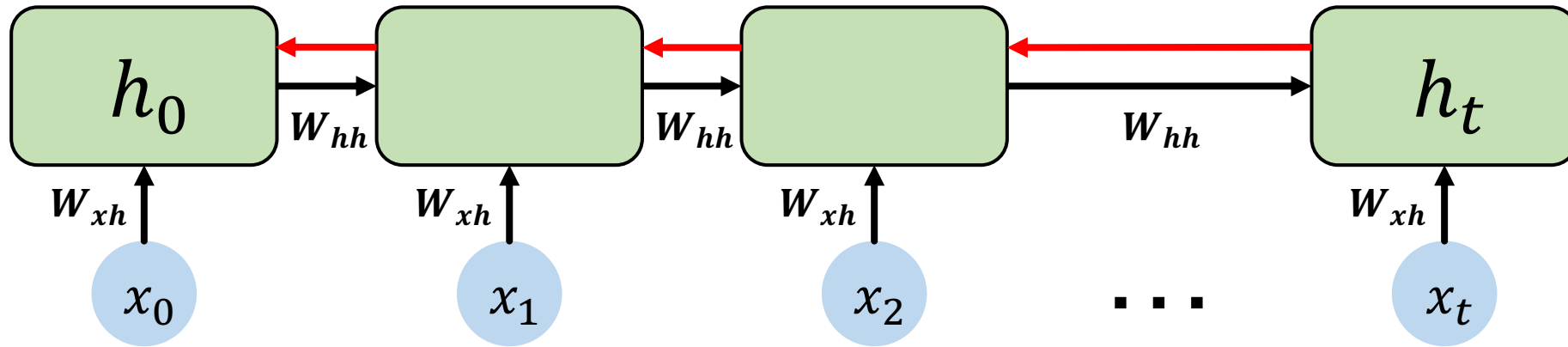
Standard RNN gradient flow: exploding gradients



Computing the gradient wrt h_0 involves **many factors of W_{hh}** (and repeated f' !)

Many values > 1 :
exploding gradients

Standard RNN gradient flow: exploding gradients

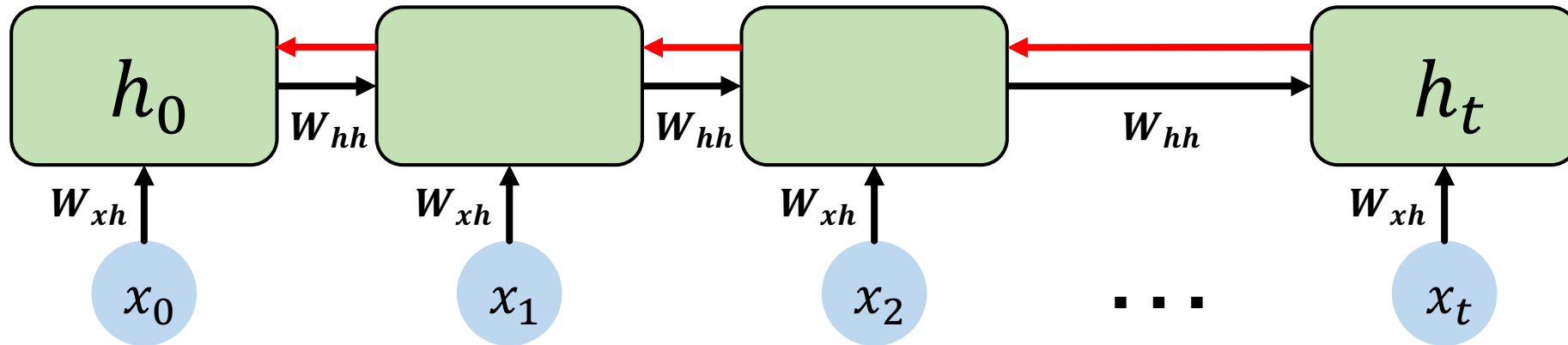


Computing the gradient wrt h_0 involves **many factors of W_{hh}** (and repeated $f'!$)

Many values > 1 :
exploding gradients

Gradient clipping to
scale big gradients

Standard RNN gradient flow: vanishing gradients



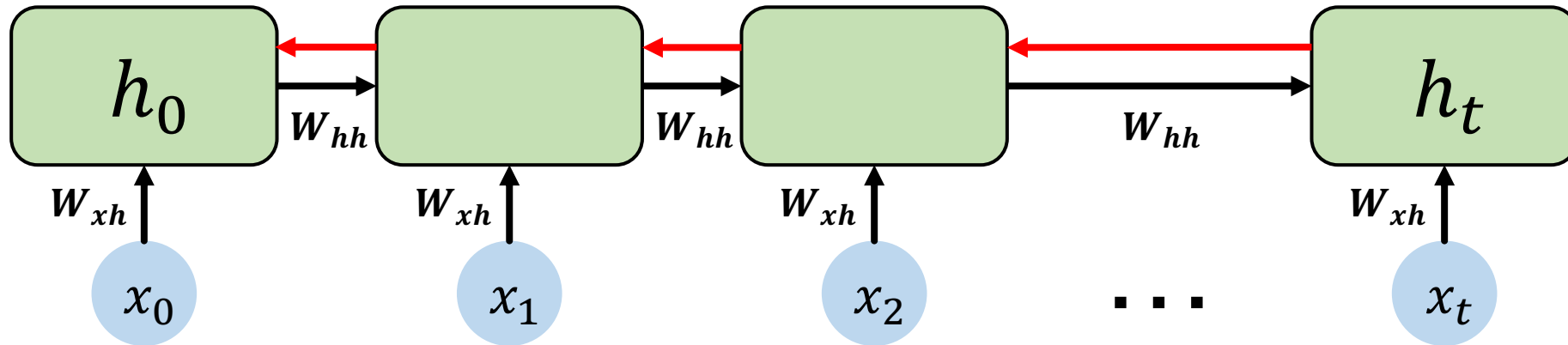
Computing the gradient wrt h_0 involves **many factors of W_{hh}** (and repeated f' !)

Many values > 1 :
exploding gradients

Gradient clipping to
scale big gradients

Many values < 1 :
vanishing gradients

Standard RNN gradient flow: vanishing gradients



Computing the gradient wrt h_0 involves **many factors of W_{hh}** (and repeated f' !)

Largest singular value > 1 :
exploding gradients

Gradient clipping to
scale big gradients

Largest singular value < 1 :
vanishing gradients

1. Activation function
2. Weight initialization
3. Network architecture

The problem of long-term dependencies

Why are vanishing gradients a problem?

The problem of long-term dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

The problem of long-term dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together



Errors due to further back time steps
have smaller and smaller gradients

The problem of long-term dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together



Errors due to further back time steps
have smaller and smaller gradients



Bias network to capture short-term
dependencies

The problem of long-term dependencies

“The clouds are in the ____”

Why are vanishing gradients a problem?

Multiply many **small numbers** together



Errors due to further back time steps
have smaller and smaller gradients



Bias network to capture short-term
dependencies

The problem of long-term dependencies

Why are vanishing gradients a problem?

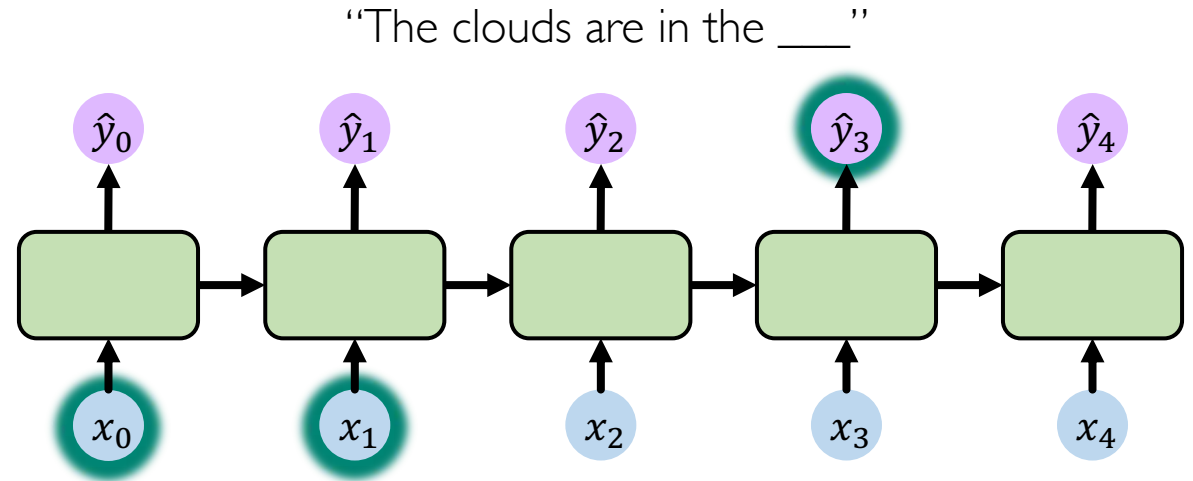
Multiply many **small numbers** together



Errors due to further back time steps
have smaller and smaller gradients



Bias parameters to capture short-term
dependencies



The problem of long-term dependencies

Why are vanishing gradients a problem?

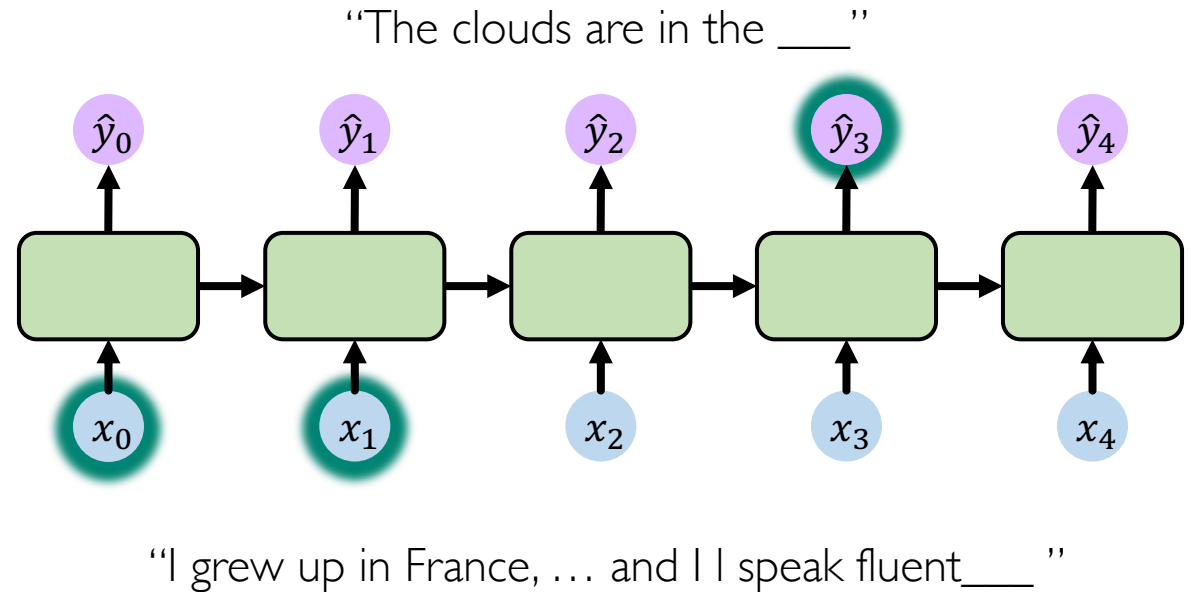
Multiply many **small numbers** together



Errors due to further back time steps
have smaller and smaller gradients



Bias parameters to capture short-term
dependencies



The problem of long-term dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together

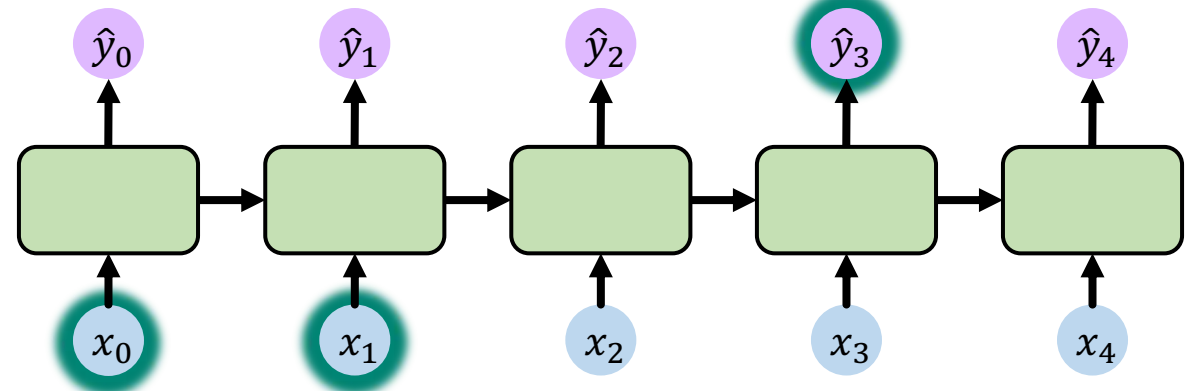


Errors due to further back time steps
have smaller and smaller gradients

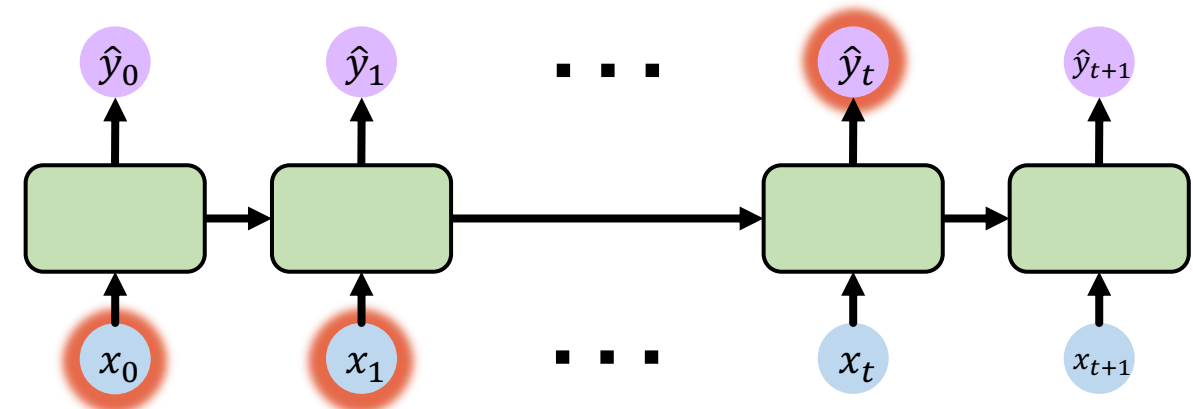


Bias parameters to capture short-term
dependencies

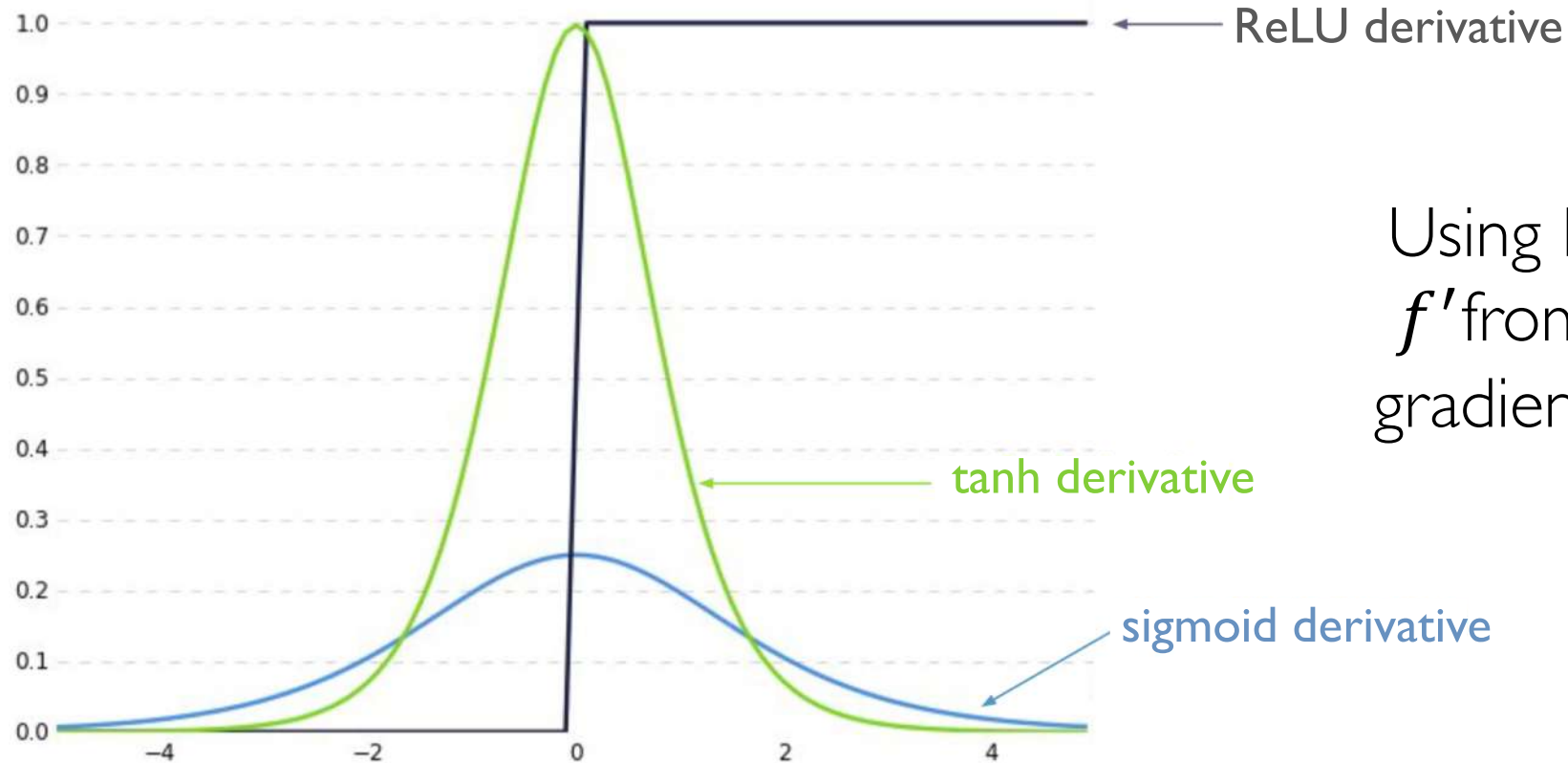
“The clouds are in the ____”



“I grew up in France, ... and I I speak fluent____”



Trick #1: activation functions



Using ReLU prevents f' from shrinking the gradients when $x > 0$

Adapted from H. Suresh, 6.S191 2018

Trick #2: parameter initialization

Initialize **weights** to identity matrix

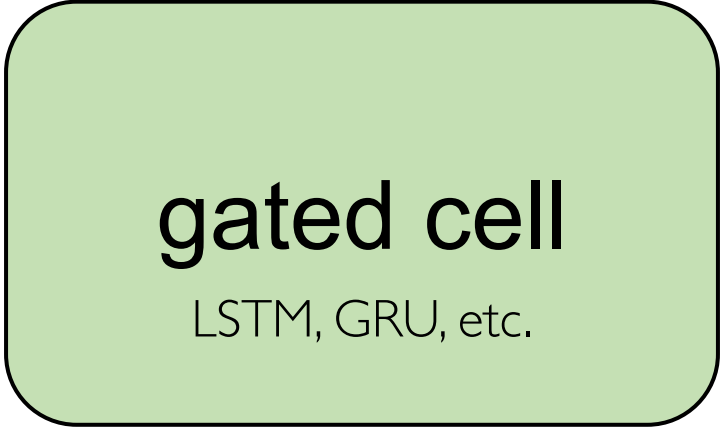
Initialize **biases** to zero

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{pmatrix}$$

This helps prevent the weights from shrinking to zero.

Solution #3: gated cells

Idea: use a more **complex recurrent unit with gates** to control what information is passed through



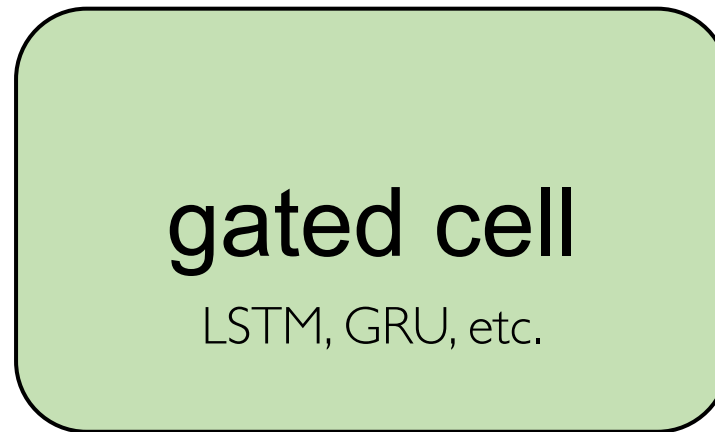
gated cell

LSTM, GRU, etc.

Adapted from H. Suresh, 6.S191 2018

Solution #3: gated cells

Idea: use a more **complex recurrent unit with gates** to control what information is passed through



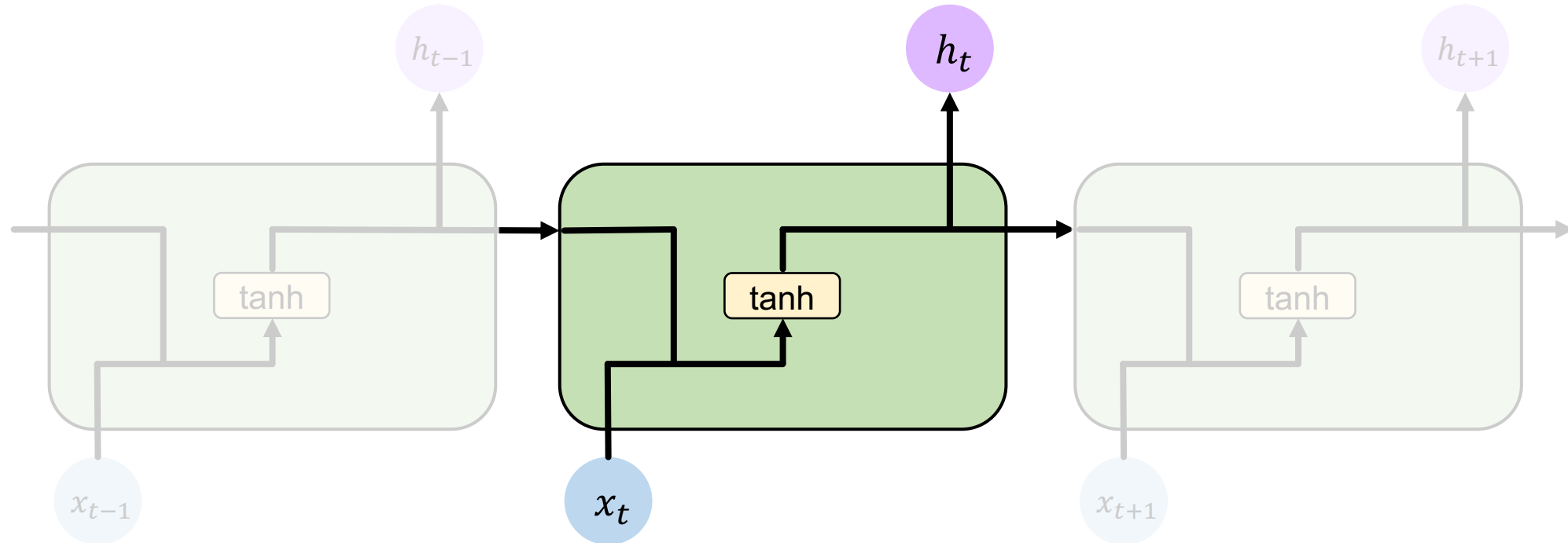
Long Short Term Memory (LSTMs) networks rely on a gated cell to track information throughout many time steps.

Adapted from H. Suresh, 6.S191 2018

Long Short Term Memory (LSTM) Networks

Standard RNN

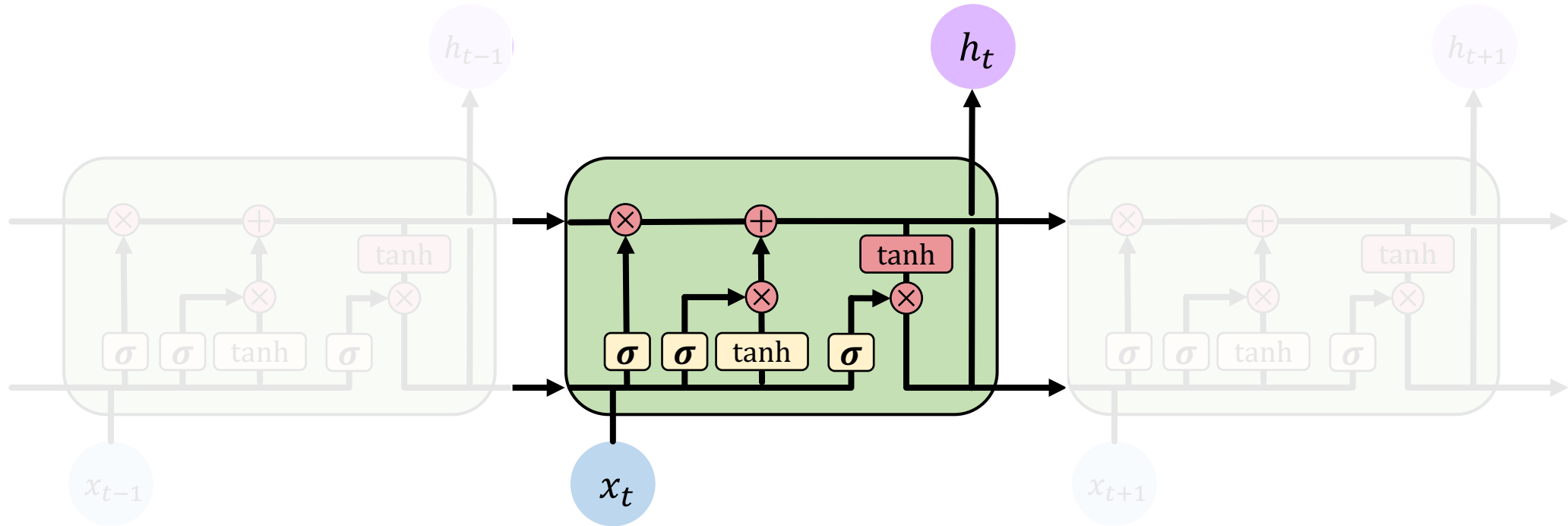
In a standard RNN, repeating modules contain a **simple computation node**



[2]

Long Short Term Memory (LSTMs)

LSTM repeating modules contain **interacting layers** that **control information flow**



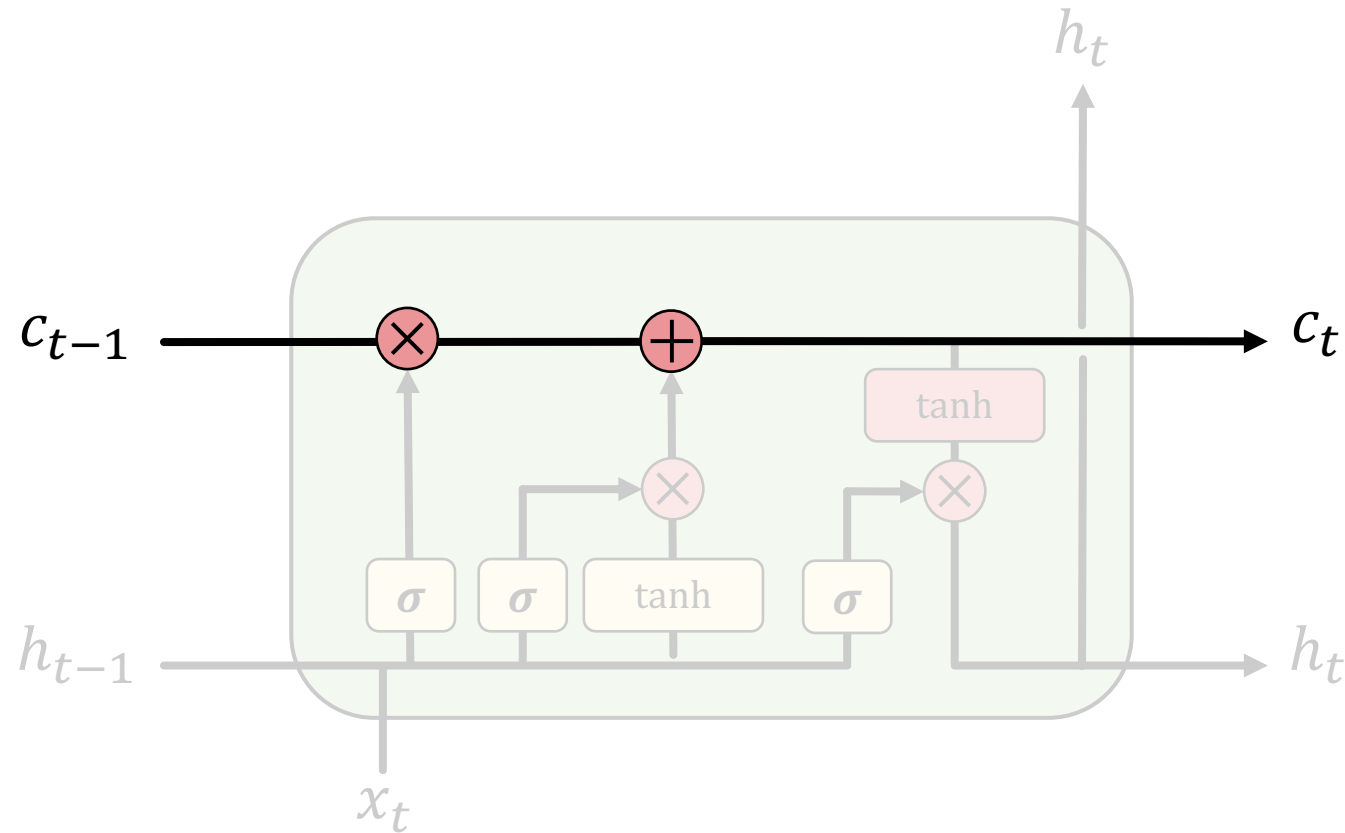
LSTM cells are able to track information throughout many timesteps

```
 tf.keras.layers.LSTM(num_units)
```

reiter & Schmidhuber, 1997 [2, 5]

Long Short Term Memory (LSTMs)

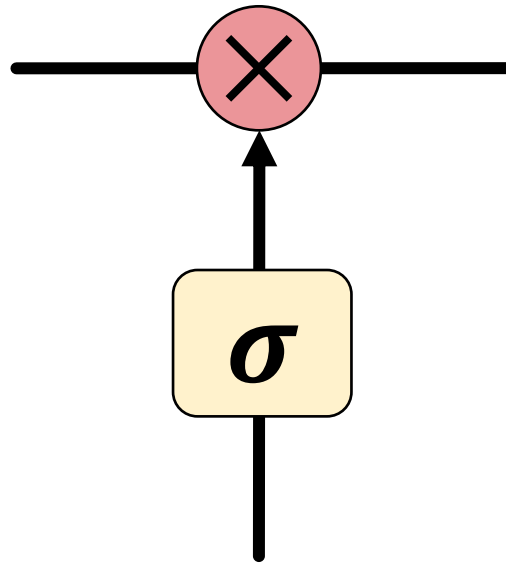
LSTMs maintain a **cell state** c_t where it's easy for information to flow



[2, 5]

Long Short Term Memory (LSTMs)

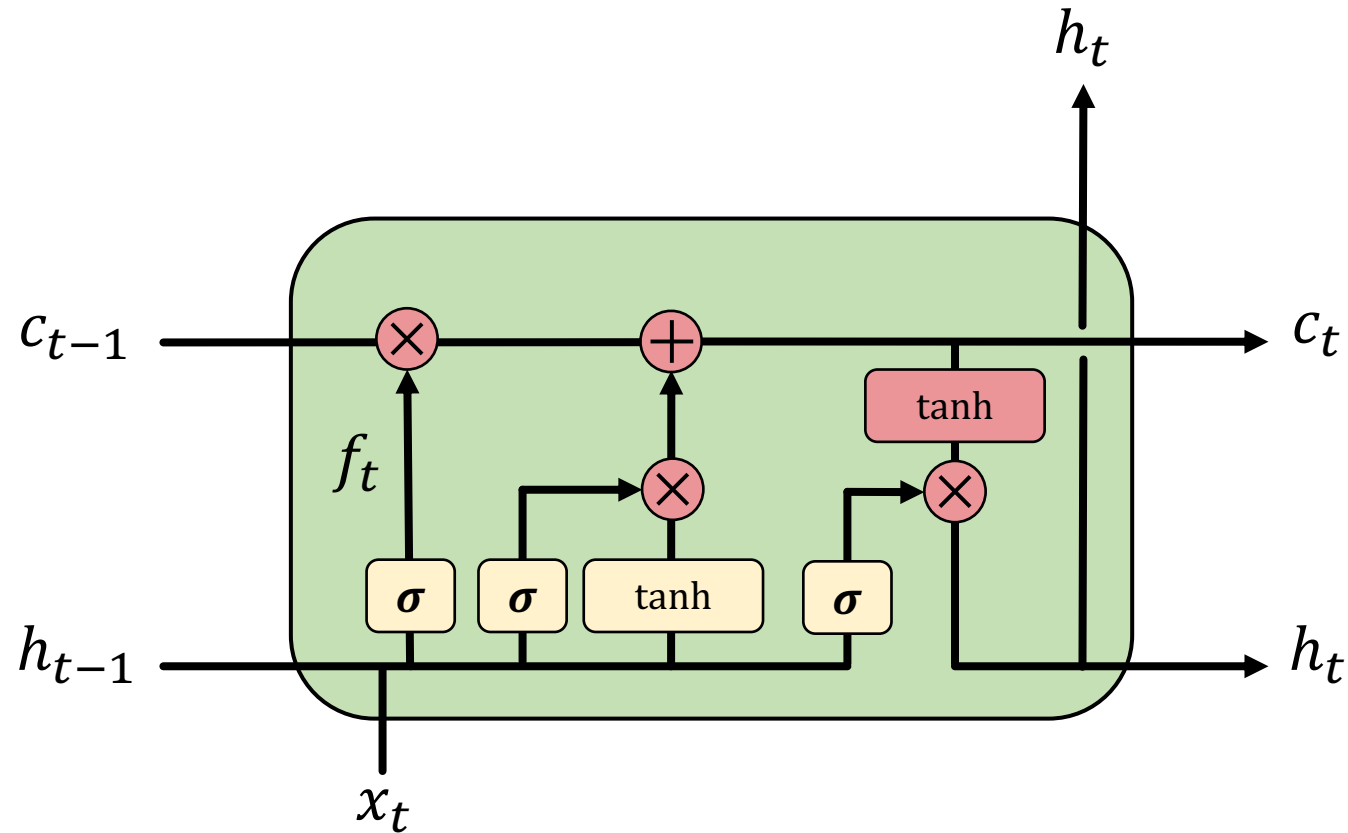
Information is **added** or **removed** to cell state through structures called **gates**



Gates optionally let information through, via a sigmoid neural net layer and pointwise multiplication

Long Short Term Memory (LSTMs)

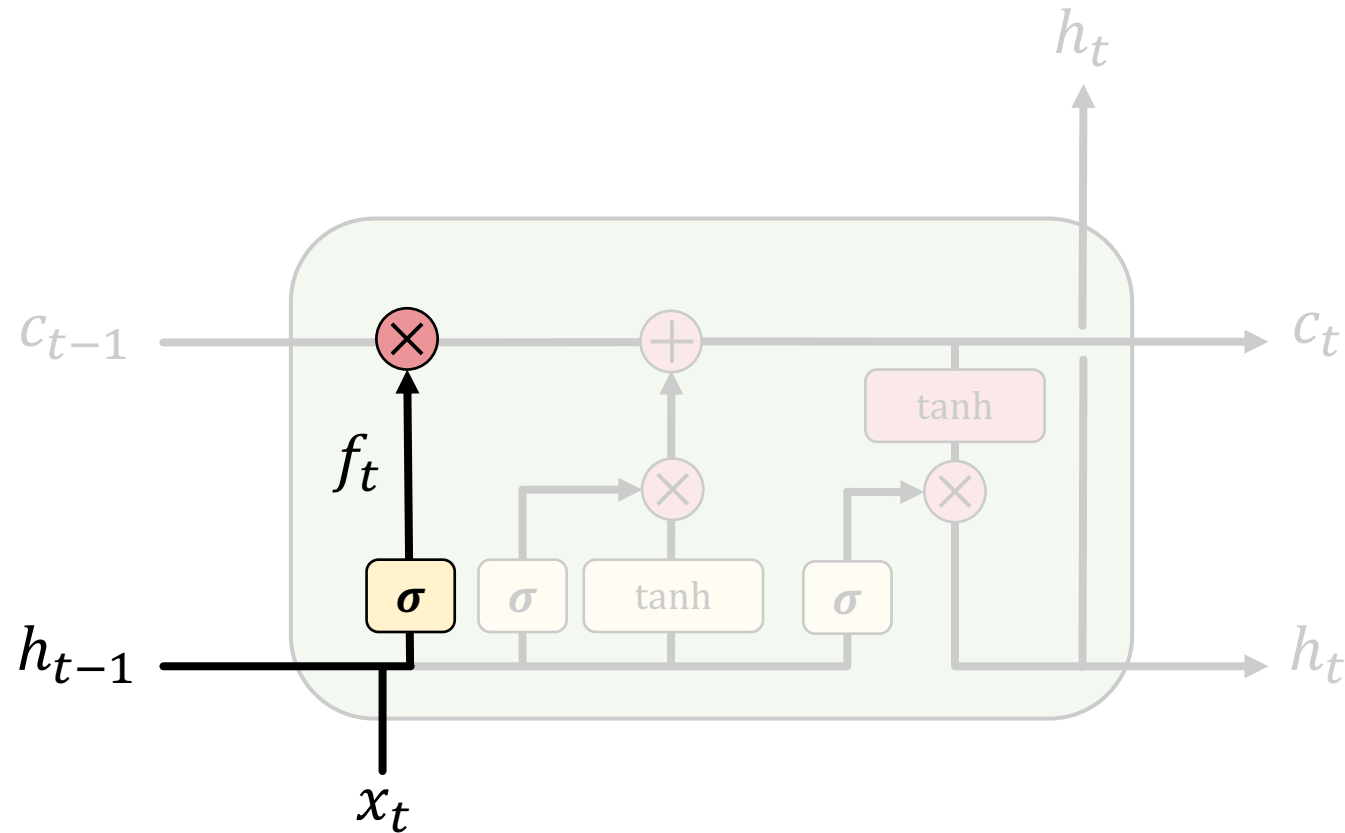
How do LSTMs work?



[2, 5]

Long Short Term Memory (LSTMs)

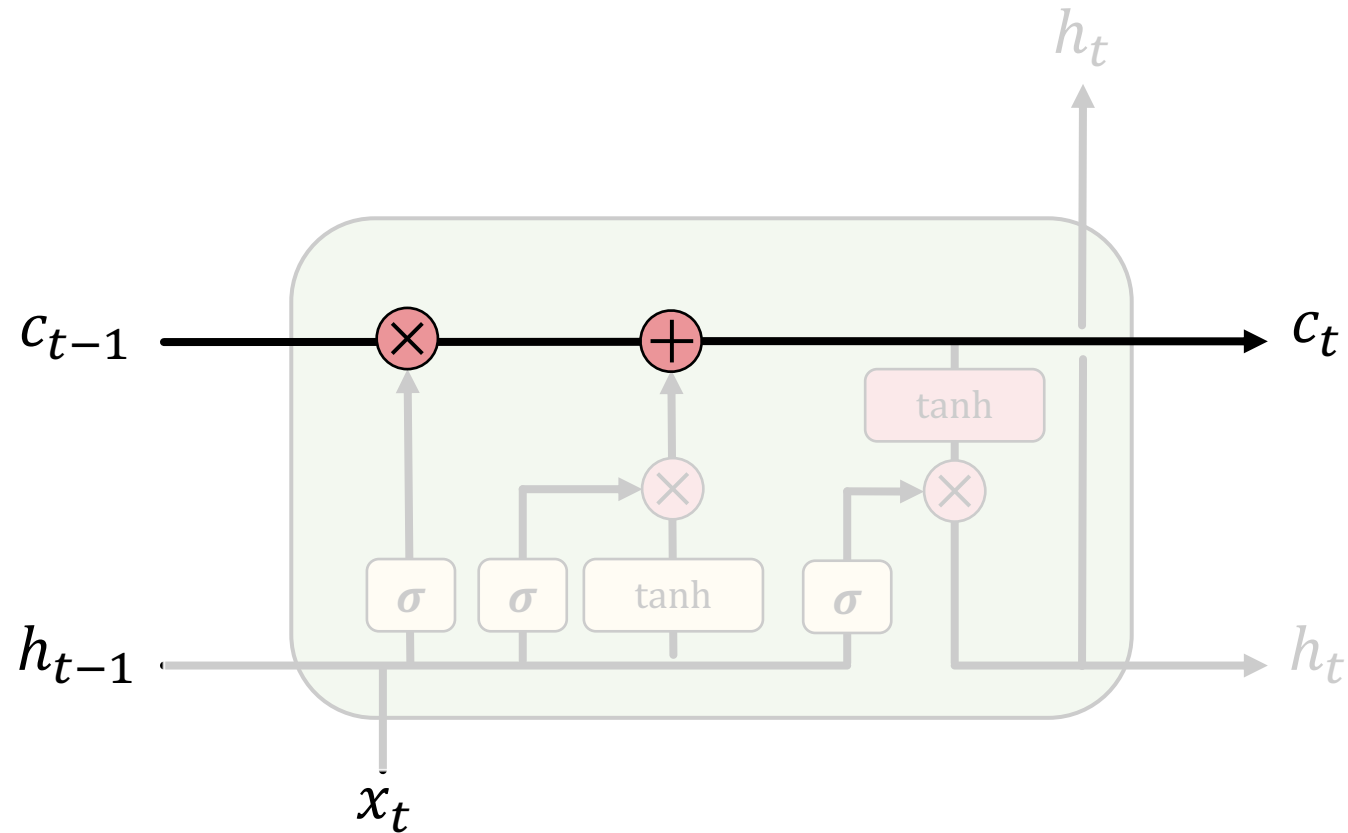
LSTMs **forget irrelevant** parts of the previous state



[2, 5]

Long Short Term Memory (LSTMs)

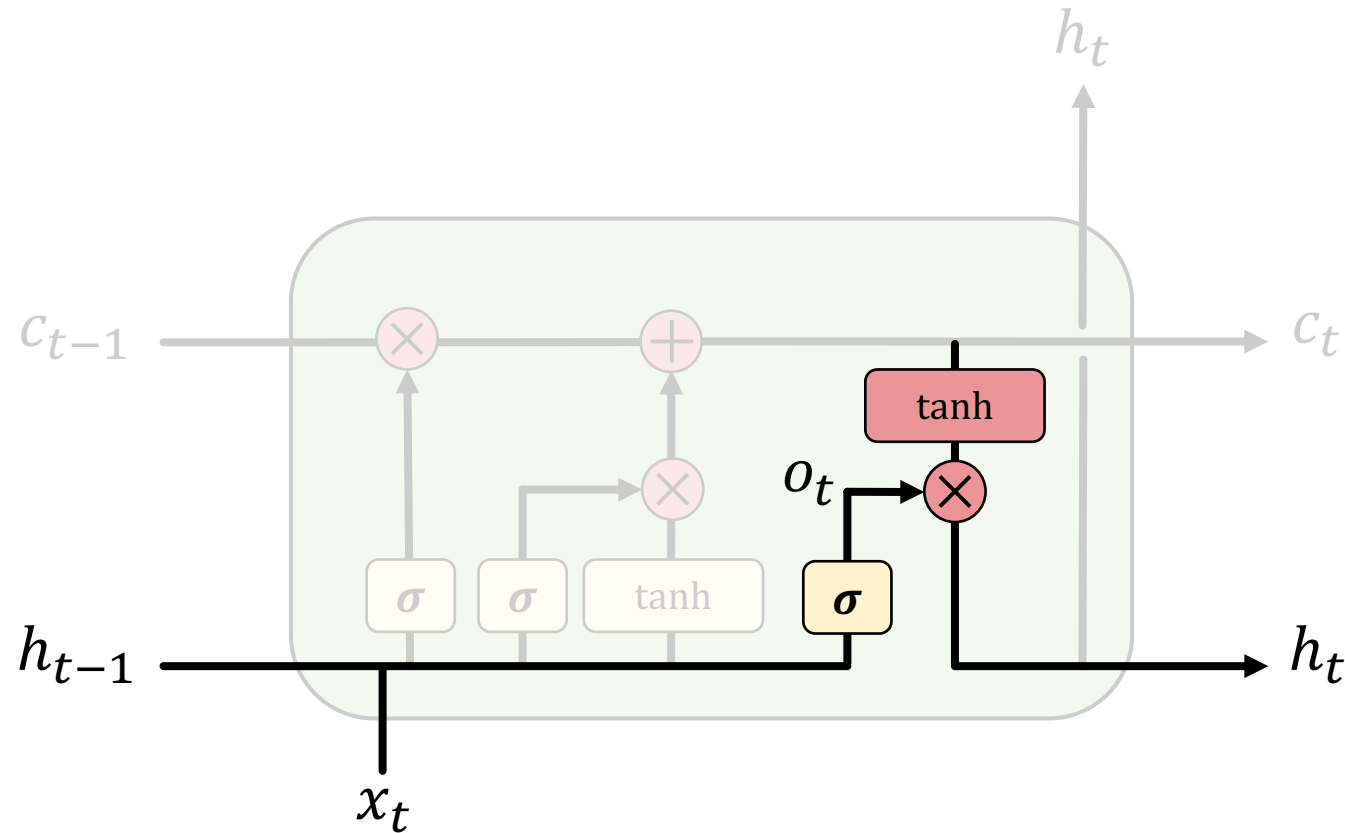
LSTMs **selectively update** cell state values



[2, 5]

Long Short Term Memory (LSTMs)

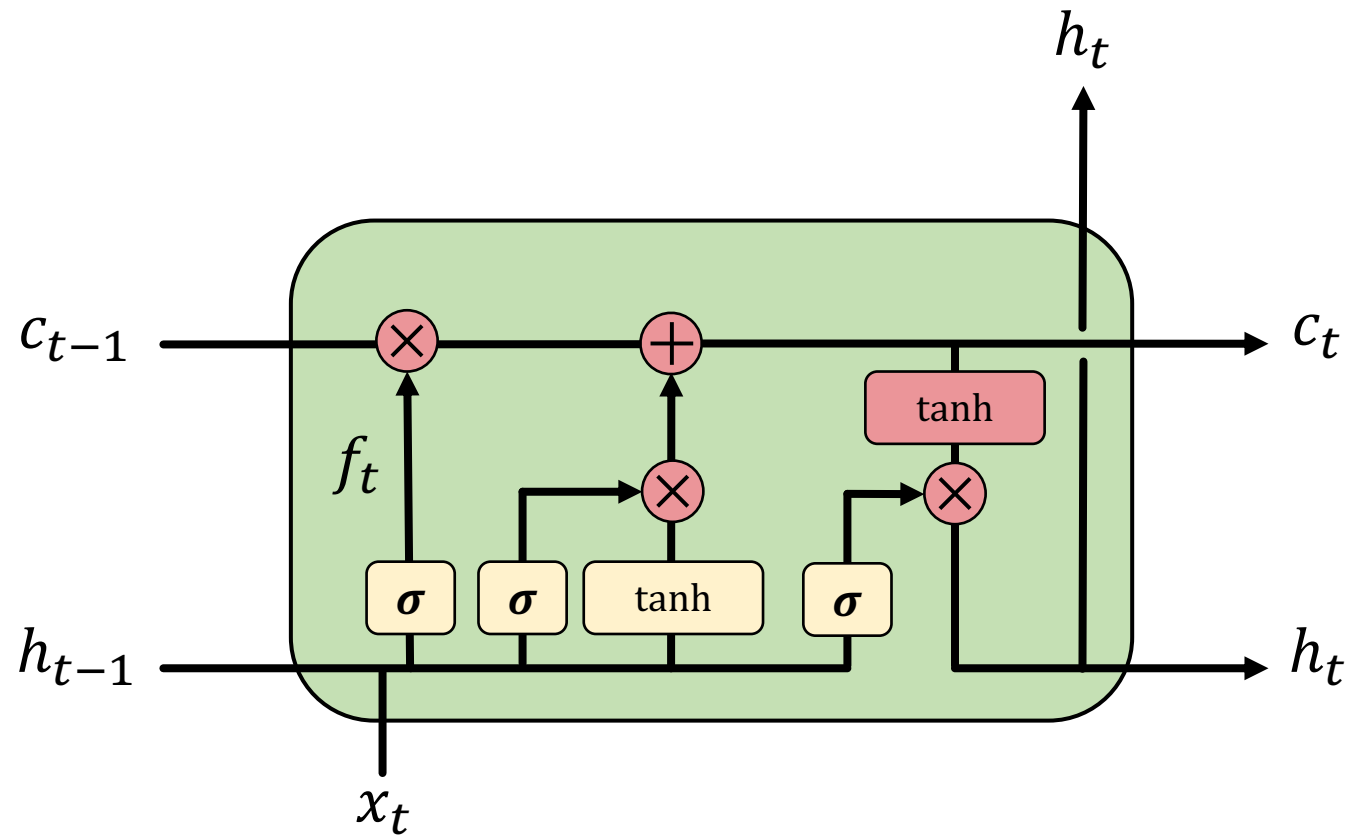
LSTMs use an **output gate** to output certain parts of the cell state



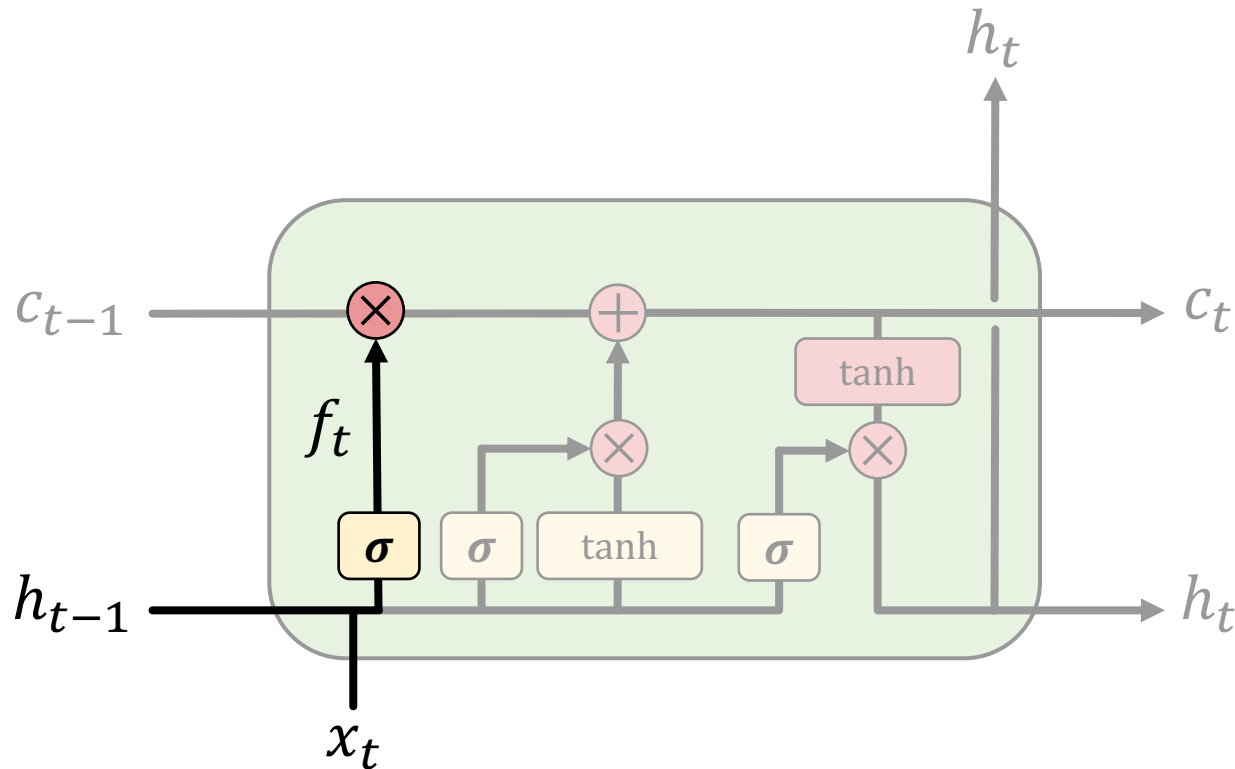
Long Short Term Memory (LSTMs)

How do LSTMs work?

1) Forget 2) Update 3) Output



LSTMs: forget irrelevant information

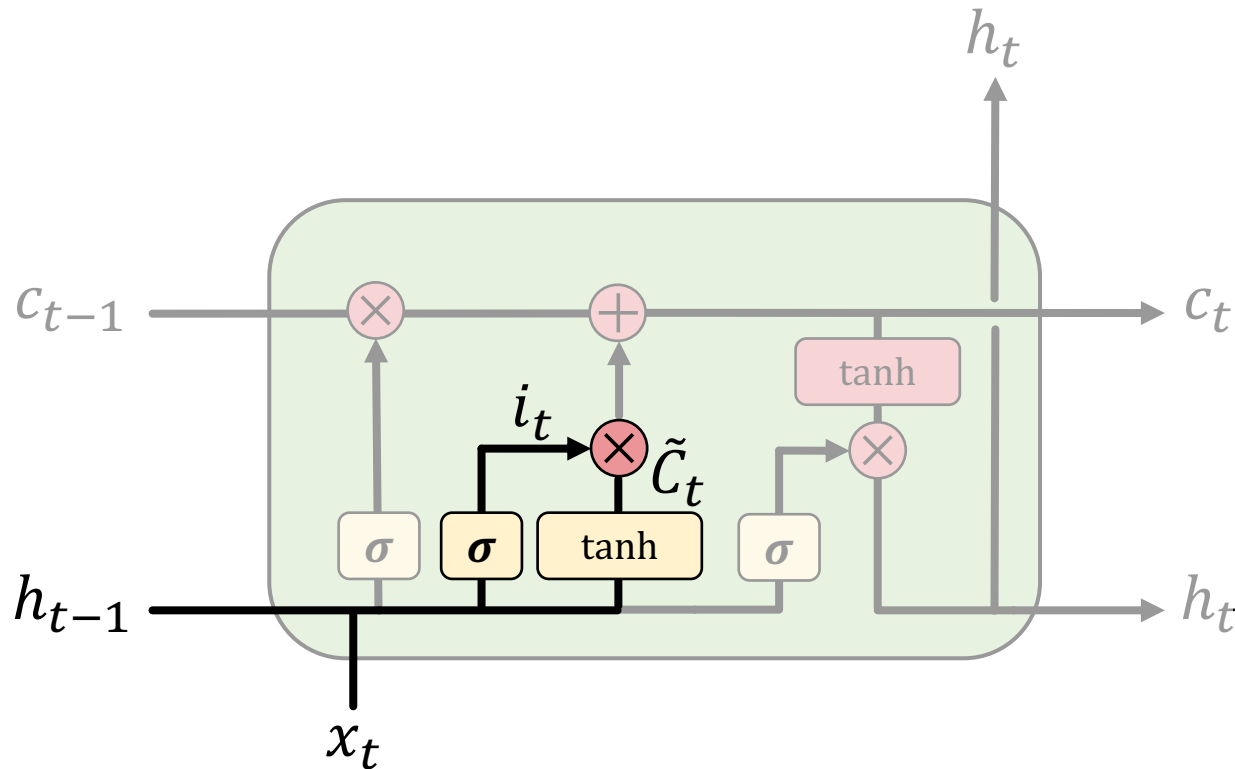


$$f_t = \sigma(W_i[h_{t-1}, x_t] + b_f)$$

- Use previous cell output and input
- Sigmoid: value 0 and 1 – “completely forget” vs. “completely keep”

ex: Forget the gender pronoun of previous subject in sentence.

LSTMs: identify new information to be stored

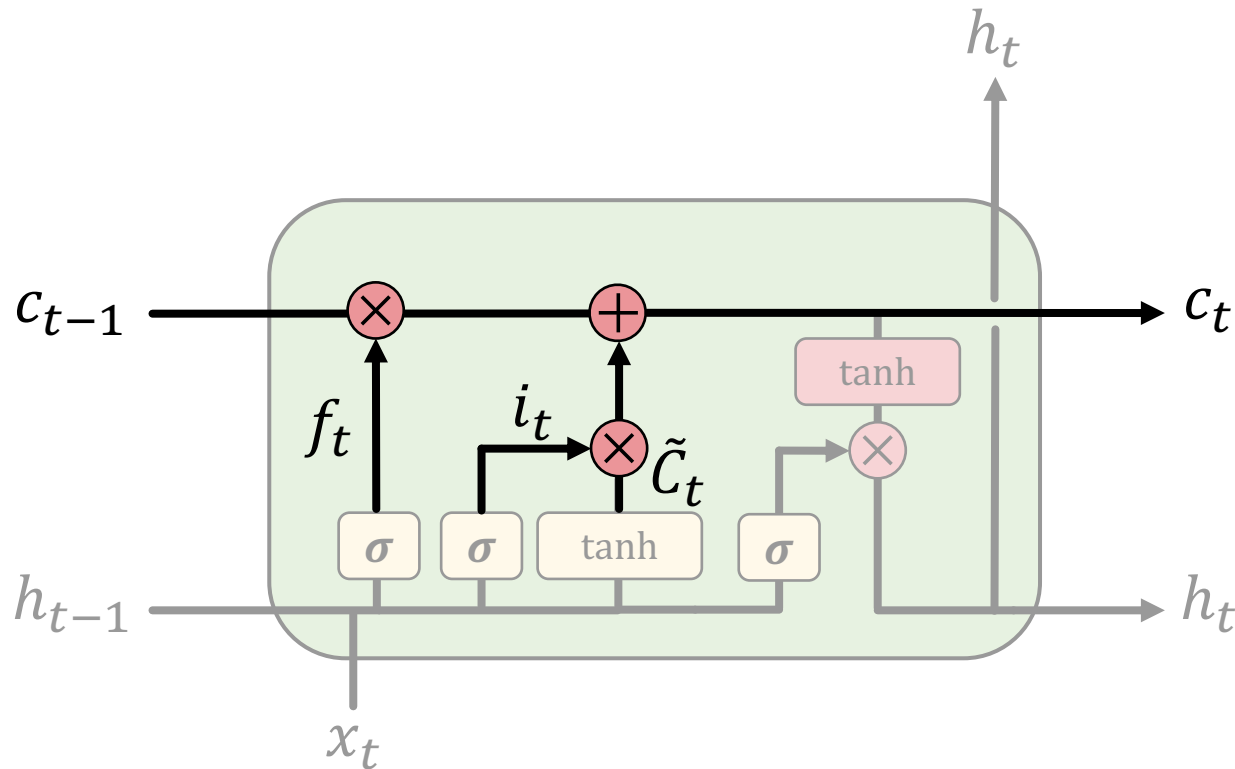


$$i_t = \sigma(\mathbf{W}_i[h_{t-1}, x_t] + b_i)$$
$$\tilde{c}_t = \tanh(\mathbf{W}_c[h_{t-1}, x_t] + b_c)$$

- Sigmoid layer: decide what values to update
- Tanh layer: generate new vector of “candidate values” that could be added to the state

ex: Add gender of new subject to replace that of old subject.

LSTMs: update cell state

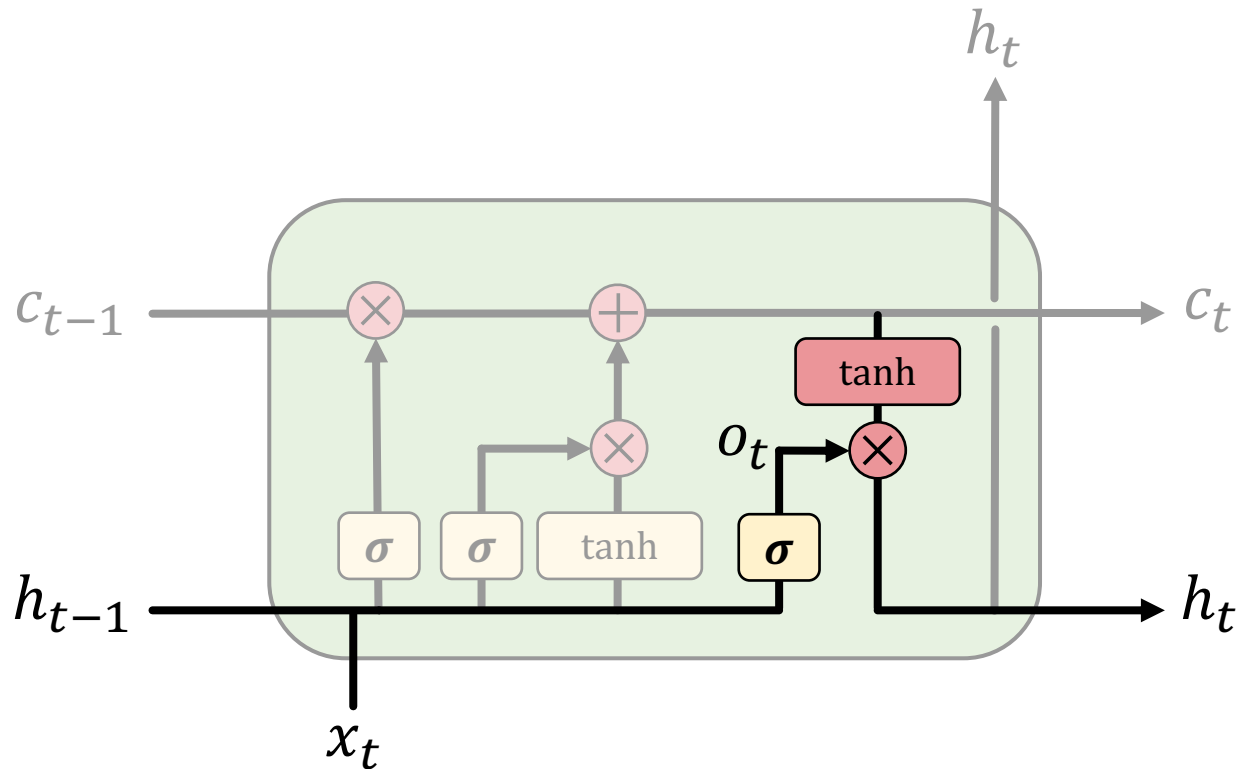


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- Apply forget operation to previous internal cell state: $f_t * C_{t-1}$
- Add new candidate values, scaled by how much we decided to update: $i_t * \tilde{C}_t$

ex: Actually drop old information and add new information about subject's gender.

LSTMs: output filtered version of cell state



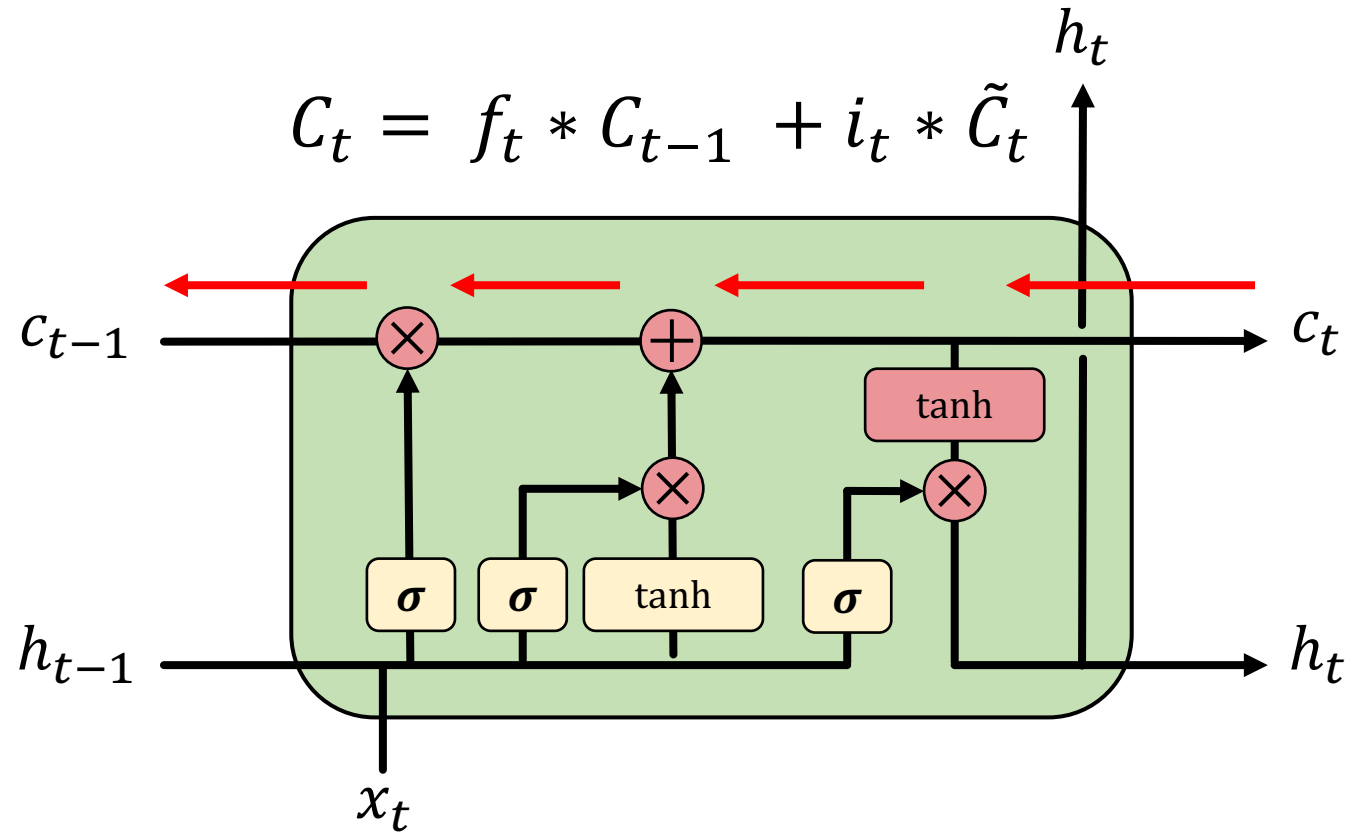
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * \tanh(C_t)$$

- Sigmoid layer: decide what parts of state to output
- Tanh layer: squash values between -1 and 1
- $o_t * \tanh(C_t)$: output filtered version of cell state

ex: Having seen a subject, may output information relating to a verb.

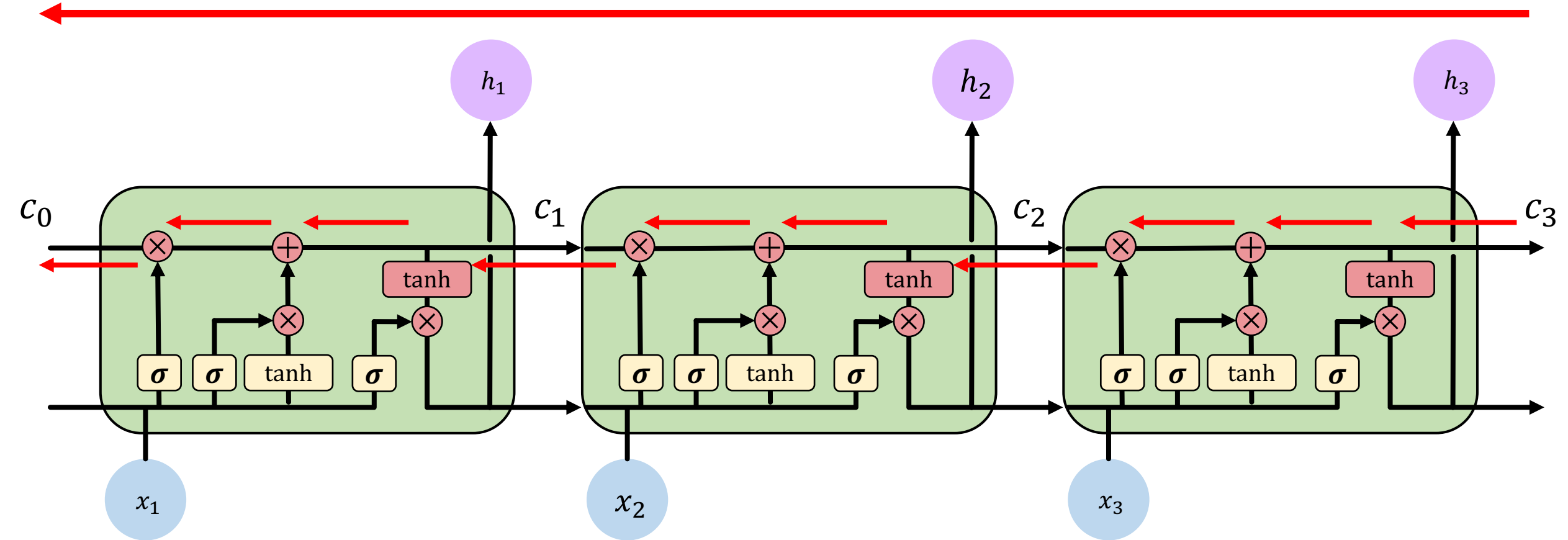
LSTM gradient flow

Backpropagation from C_t to C_{t-1} requires only elementwise multiplication!
No matrix multiplication $\rightarrow$ avoid vanishing gradient problem.



LSTM gradient flow

Uninterrupted gradient flow!



LSTMs: key concepts

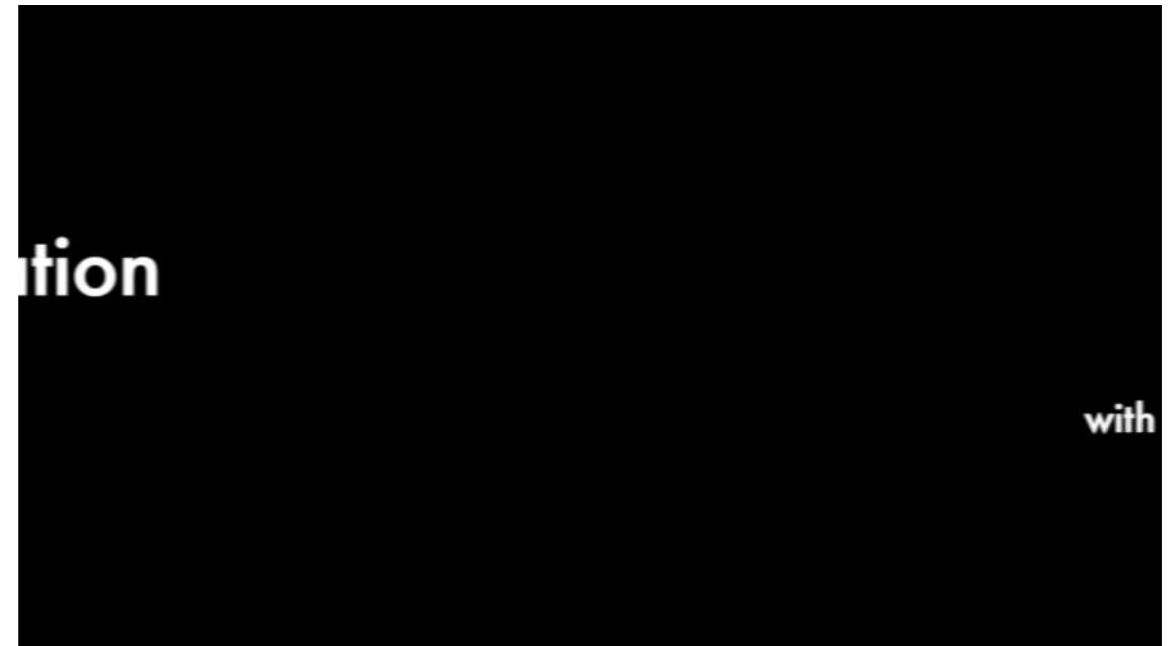
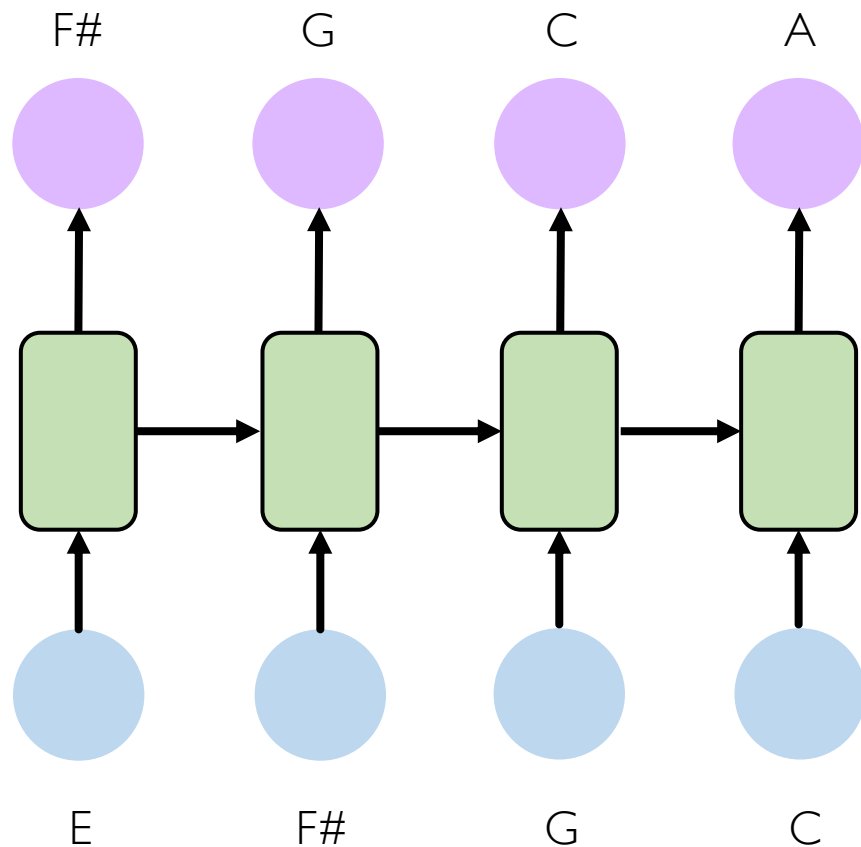
1. Maintain a **separate cell state** from what is outputted
2. Use **gates** to control the **flow of information**
 - Forget gate gets rid of irrelevant information
 - Selectively update cell state
 - Output gate returns a filtered version of the cell state
3. Backpropagation from c_t to c_{t-1} doesn't require matrix multiplication:
uninterrupted gradient flow

RNN Applications

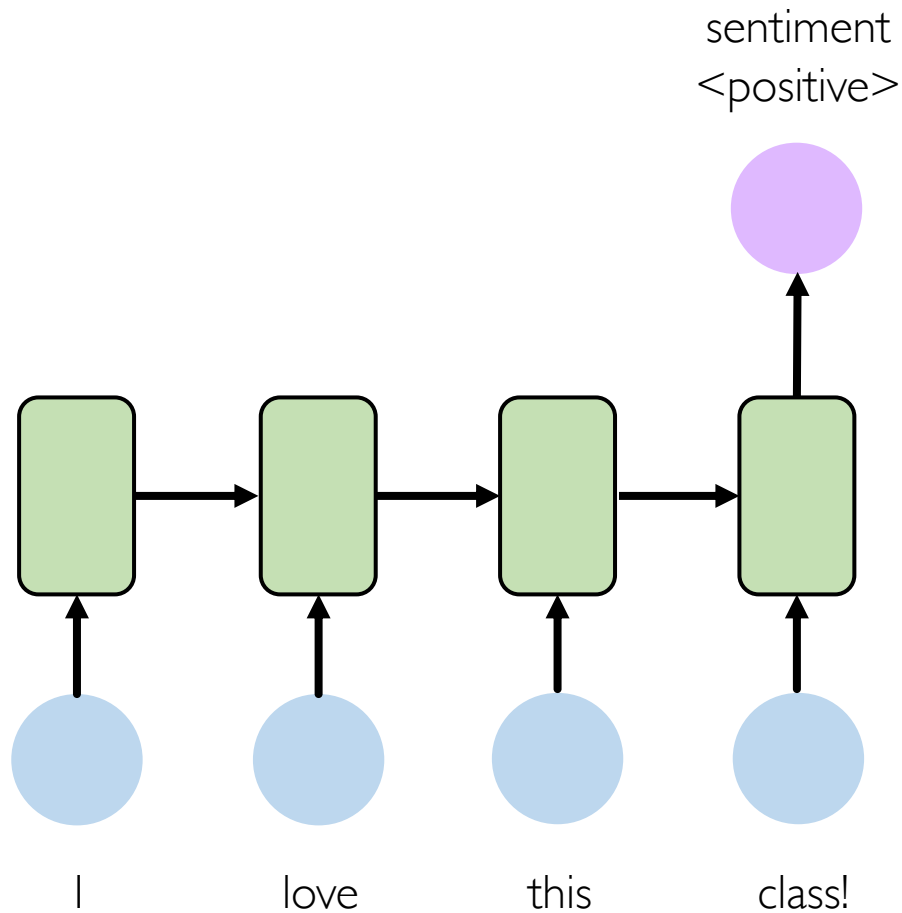
Example task: music generation

Input: sheet music

Output: next character in sheet music



Example task: sentiment classification

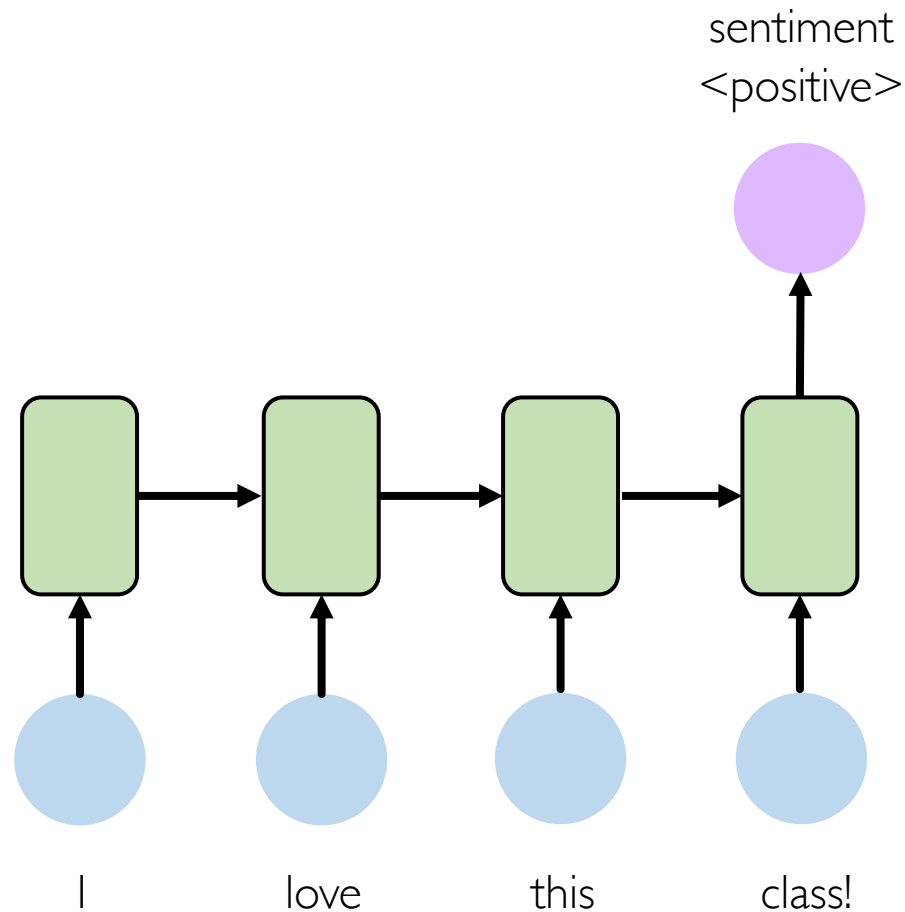


Input: sequence of words

Output: probability of having positive sentiment

```
 loss = tf.nn.softmax_cross_entropy_with_logits(  
    labels=model.y, logits=model.pred  
)
```

Example task: sentiment classification

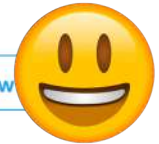


Tweet sentiment classification



Ivar Hagendoorn
@IvarHagendoorn

Follow



The @MIT Introduction to #DeepLearning is definitely one of the best courses of its kind currently available online
introtodeeplearning.com

12:45 PM - 12 Feb 2018



Angels-Cave
@AngelsCave

Follow

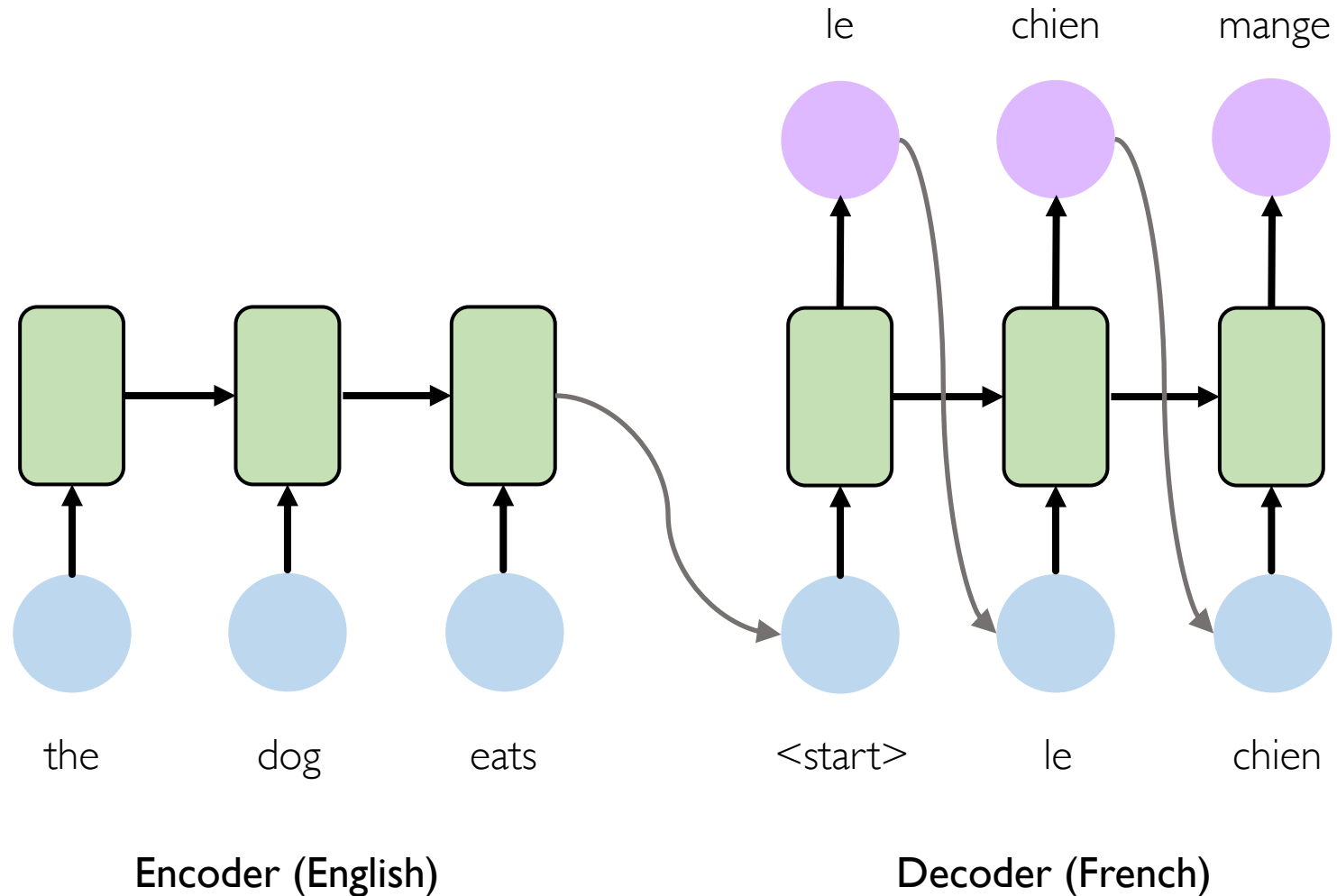


Replying to @Kazuki2048

I wouldn't mind a bit of snow right now. We haven't had any in my bit of the Midlands this winter! :(

2:19 AM - 25 Jan 2019

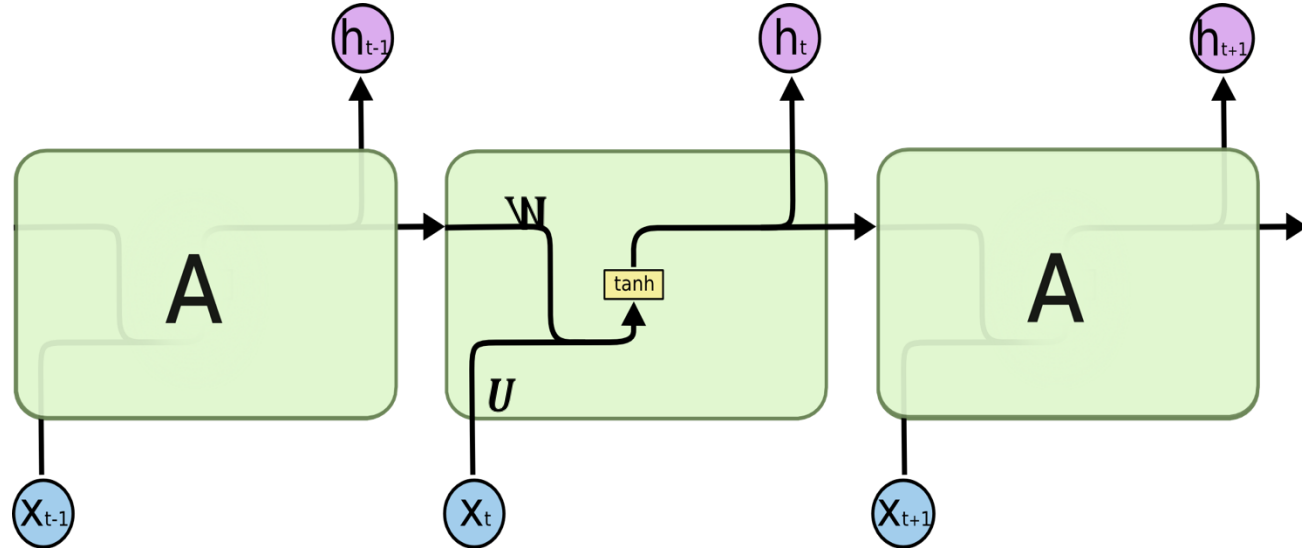
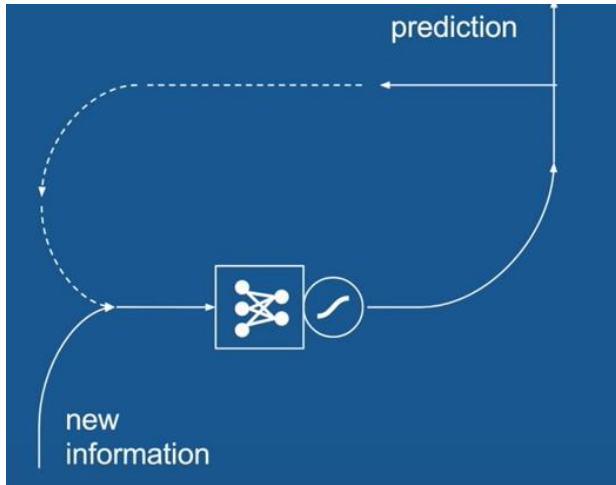
Example task: machine translation



Encoder (English)

Decoder (French)

The standard RNN module



$$h_t = f(Ux_t + Wh_{t-1} + b)$$

new weight = weight - learning rate*gradient

$$2.0999 = 2.1$$

Not much of a difference

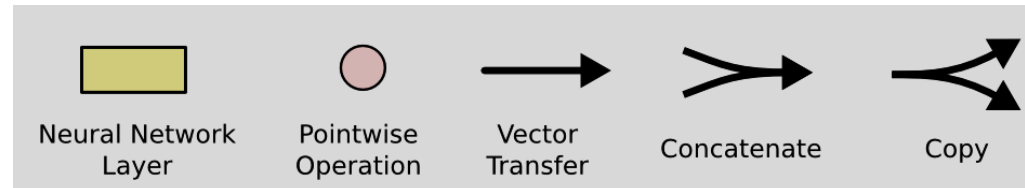
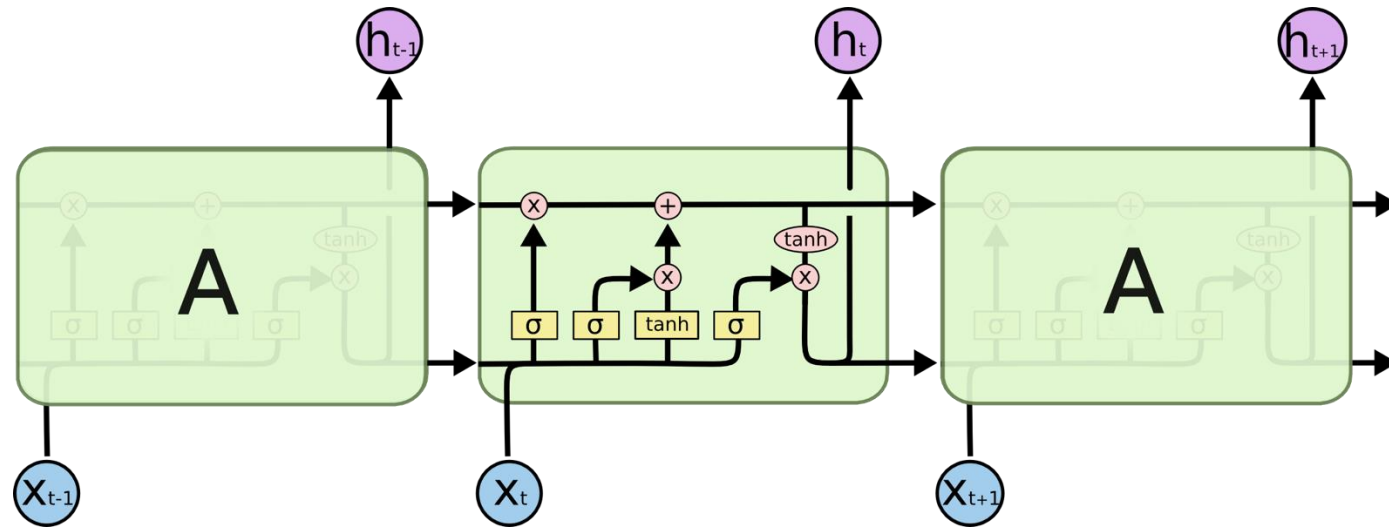
$$0.001$$

update value

Gradient vanishing issue: by the end of the RNN, the data from the first timestep has very little impact on the output of the RNN.

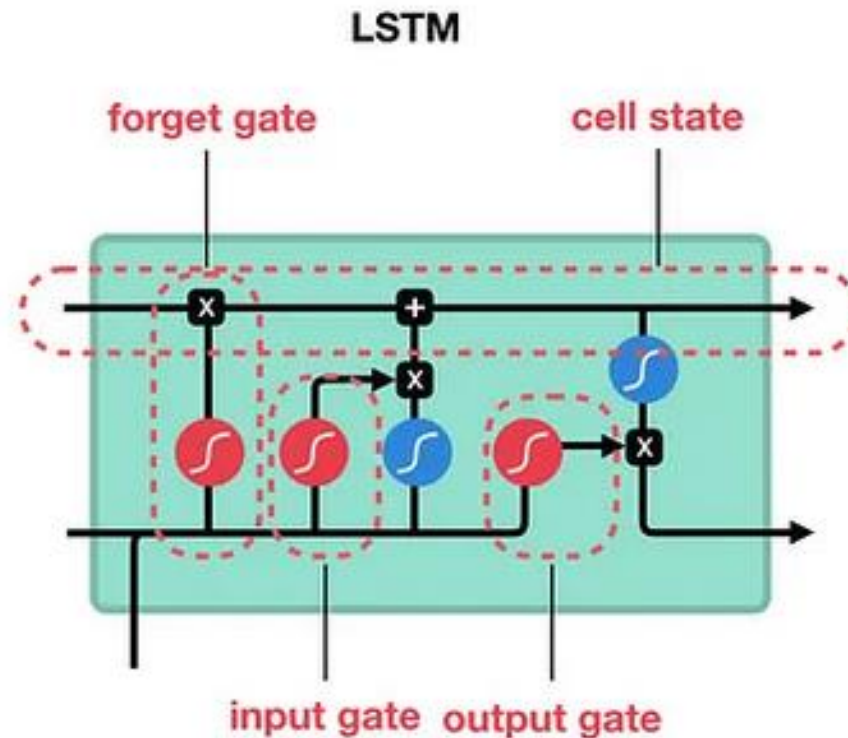
The long-short term memory (LSTM) module

LSTMs are explicitly designed to avoid the long-term dependency problem.



The long-short term memory (LSTM) module

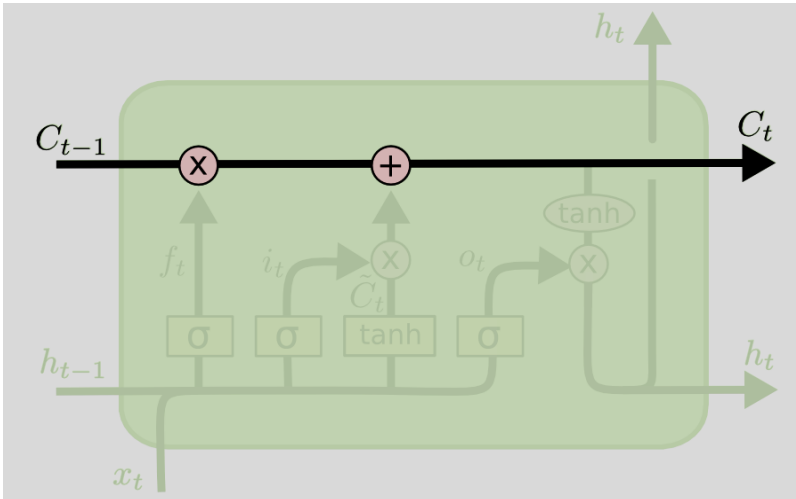
LSTMs are explicitly designed to avoid the long-term dependency problem.



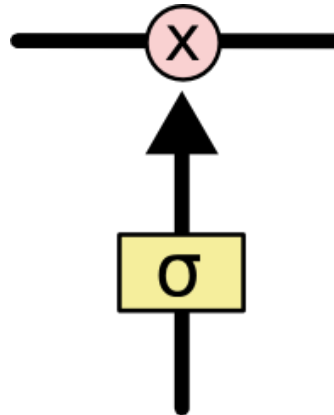
Two keys of LSTM

“Cell state” which works like a conveyor belt runs straight down the entire chain, easy for information to flow along without changes.

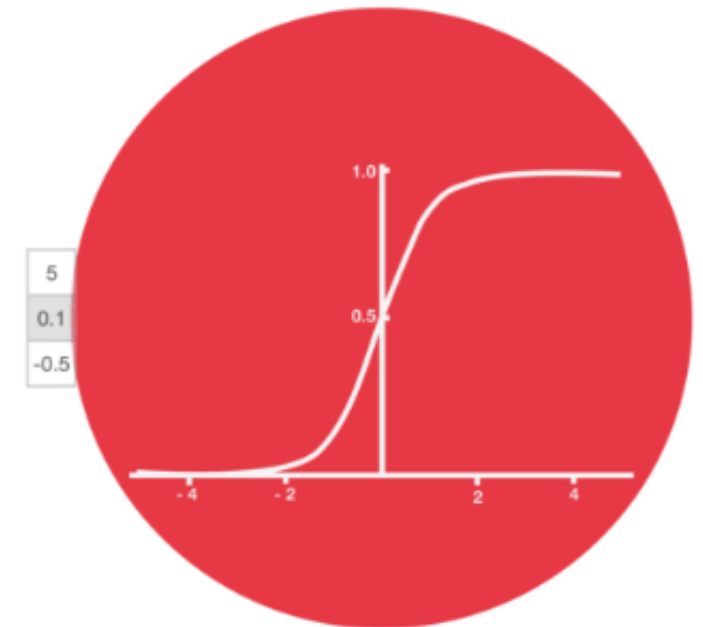
“Gates” which control or decide what kind of information could go or throw away from the cell state.



Cell state



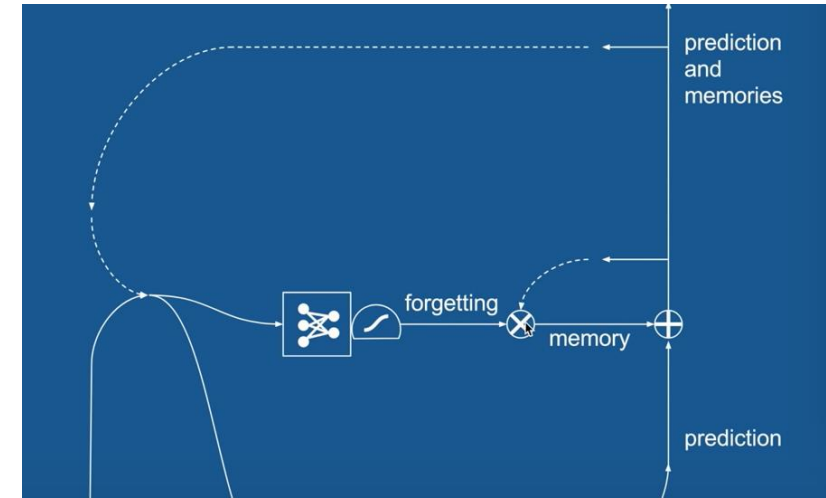
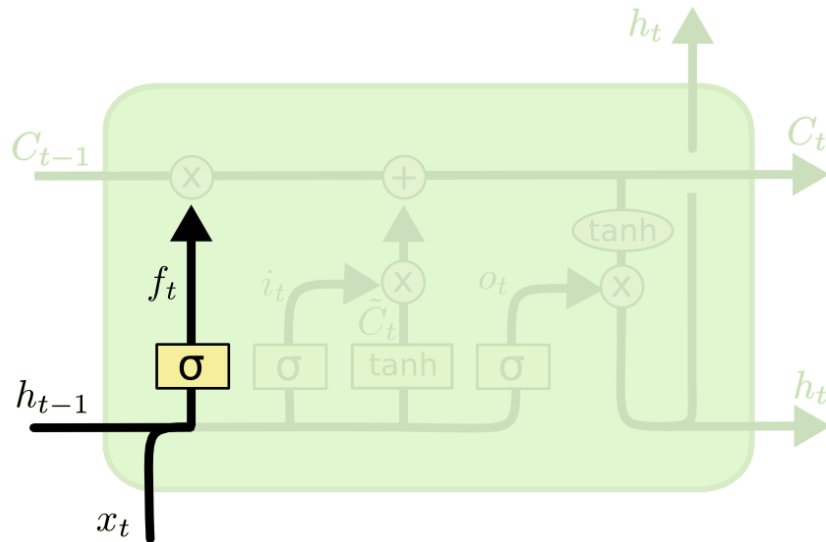
Gate



Sigmoid squishes values to be between 0 and 1

LSTM – forget gate

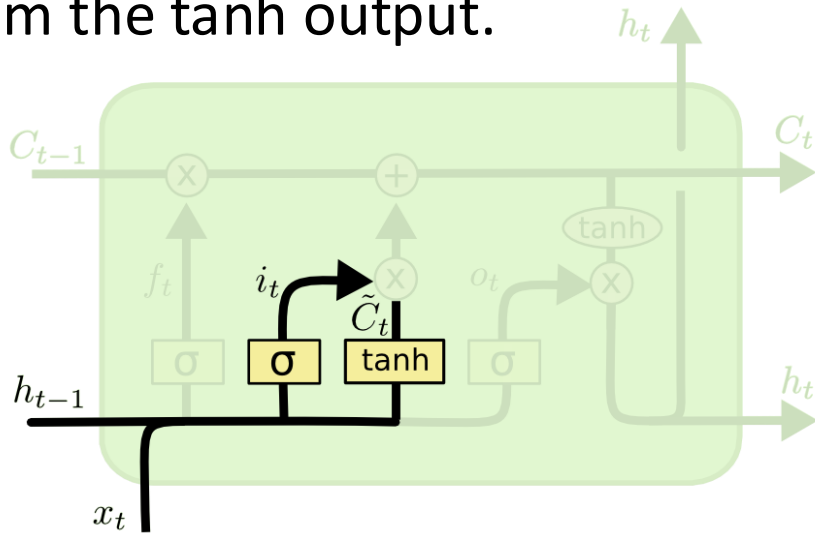
This gate decides what information should be thrown away or kept. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

LSTM – input gate

First, we pass the previous hidden state and current input into a sigmoid function. That decides which values will be updated by transforming the values to be between 0 and 1. 0 means not important, and 1 means important. You also pass the hidden state and current input into the tanh function to squish values between -1 and 1 to help regulate the network. Then you multiply the tanh output with the sigmoid output. The sigmoid output will decide which information is important to keep from the tanh output.

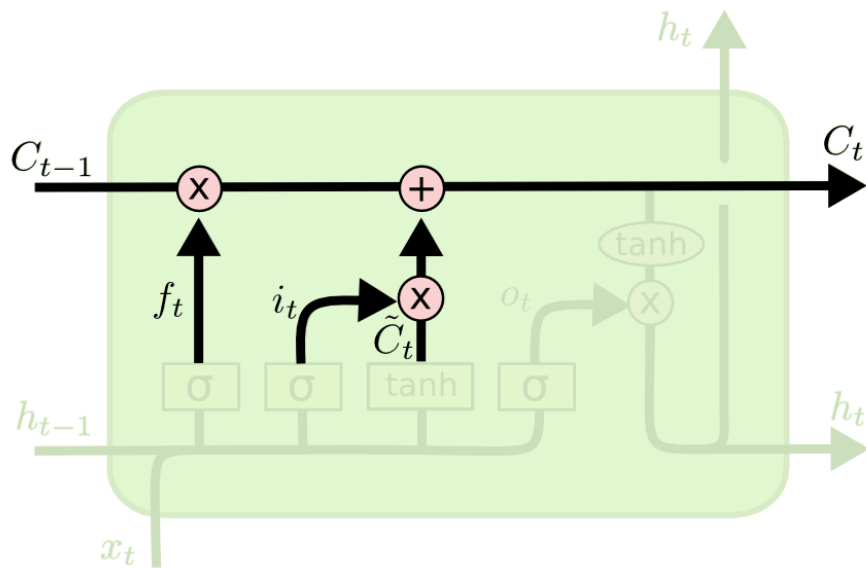


$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM – cell state update

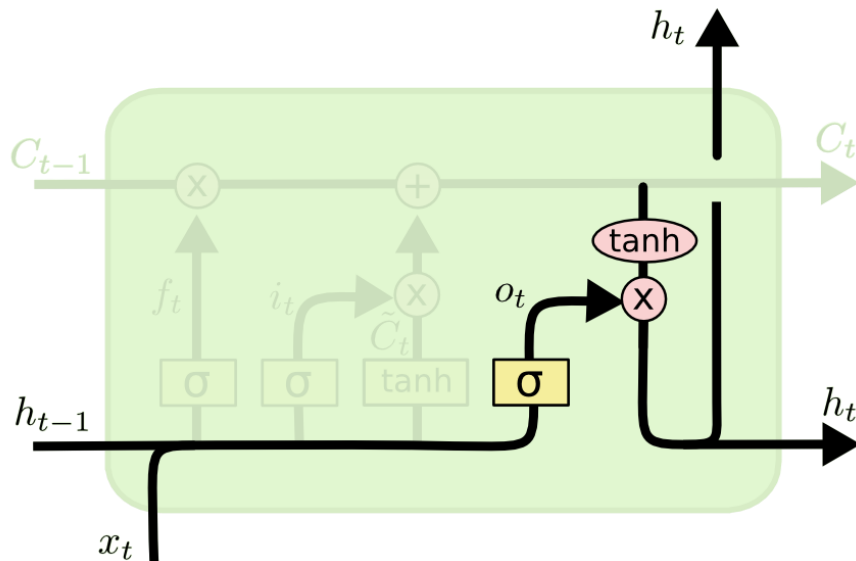
First, the cell state gets pointwise multiplied by the forget vector. This has a possibility of dropping values in the cell state if it gets multiplied by values near 0. Then we take the output from the input gate and do a pointwise addition which updates the cell state to new values that the neural network finds relevant.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM – output gate

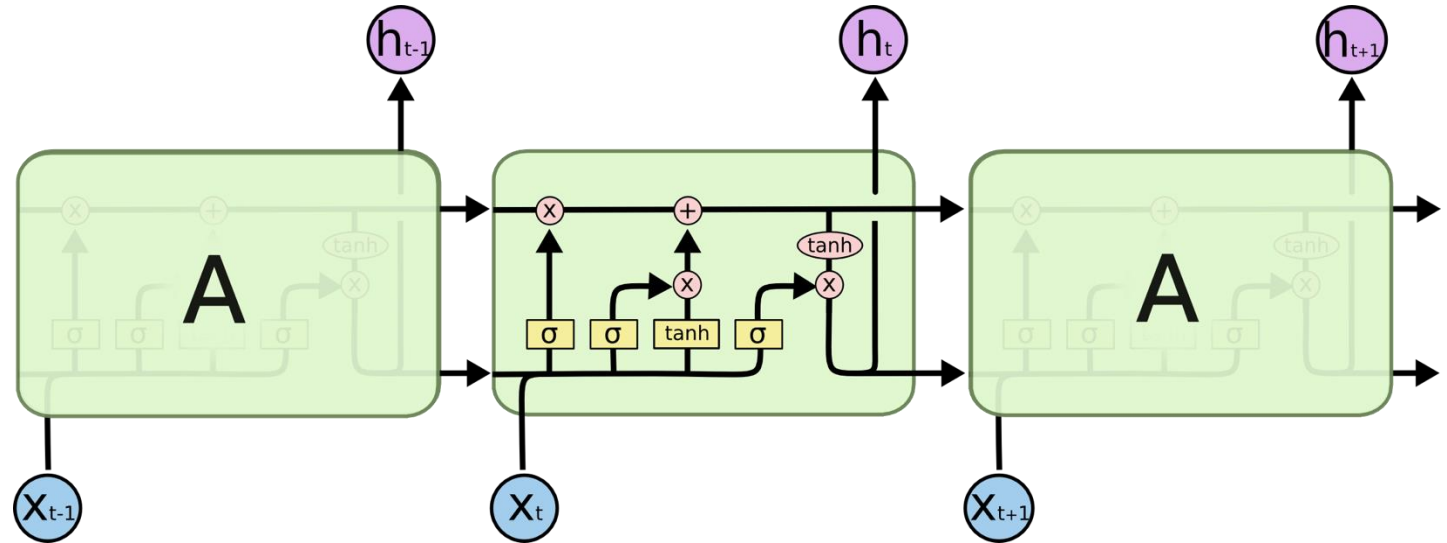
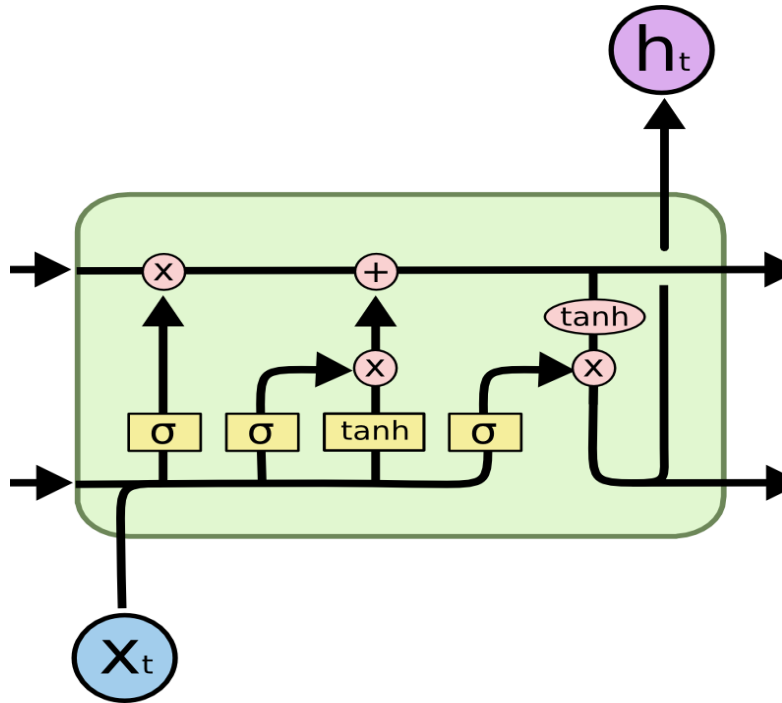
First, we pass the previous hidden state and the current input into a sigmoid function. Then we pass the newly modified cell state to the tanh function. We multiply the tanh output with the sigmoid output to decide what information the hidden state should carry. The output is the hidden state.



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

LSTM - revisit



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Code demo

```
def LSTMCELL(prev_ct, prev_ht, input):  
    combine = prev_ht + input  
    ft = forget_layer(combine)  
    candidate = candidate_layer(combine)  
    it = input_layer(combine)  
    Ct = prev_ct * ft + candidate * it  
    ot = output_layer(combine)  
    ht = ot * tanh(Ct)  
    return ht, Ct
```

```
ct = [0, 0, 0]  
ht = [0, 0, 0]
```

```
for input in inputs:  
    ct, ht = LSTMCELL(ct, ht, input)
```

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

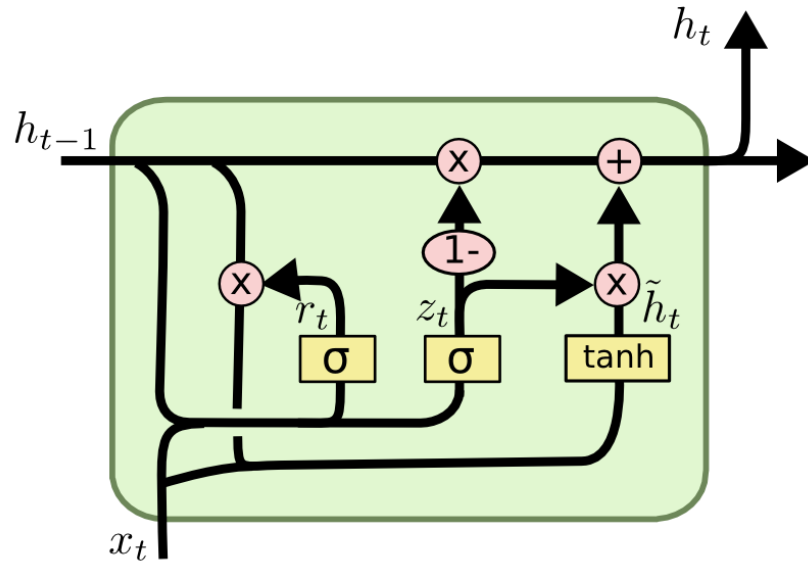
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

The gated recurrent units (GRUs) module



$$z_t = \sigma (W_z \cdot [h_{t-1}, x_t])$$

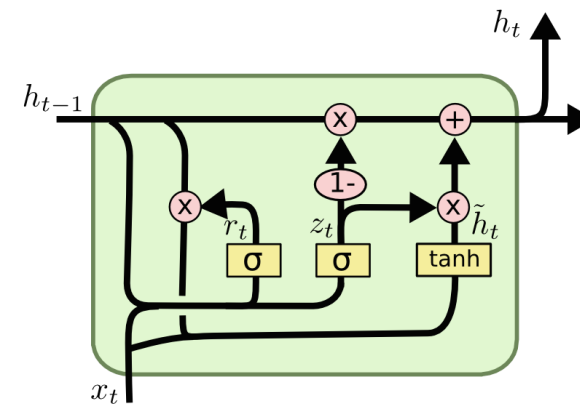
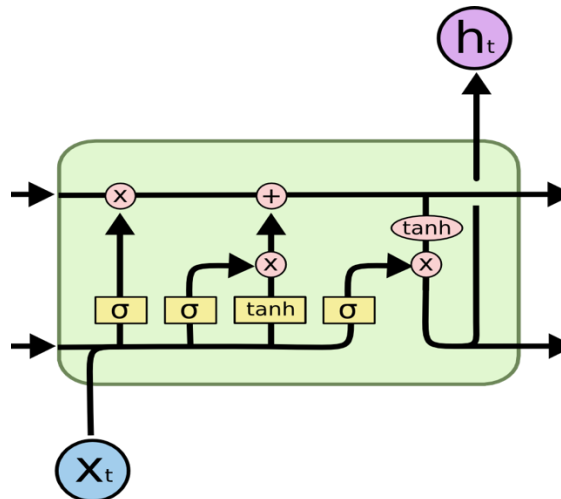
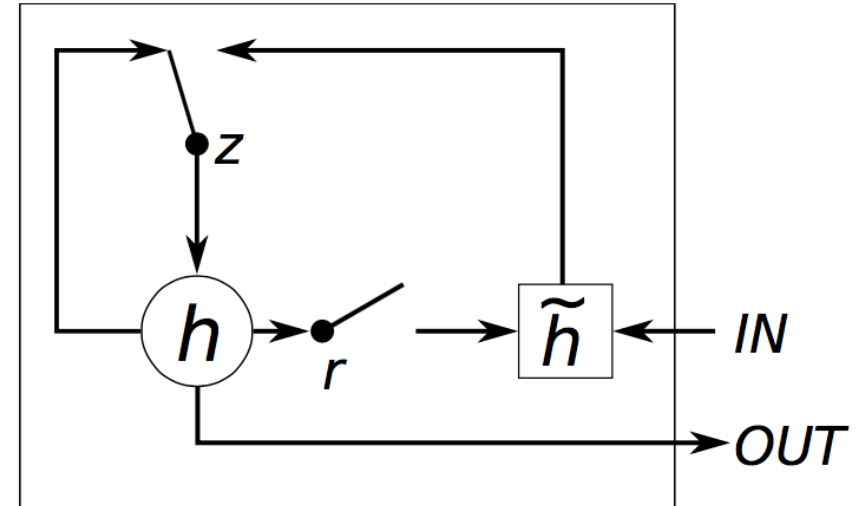
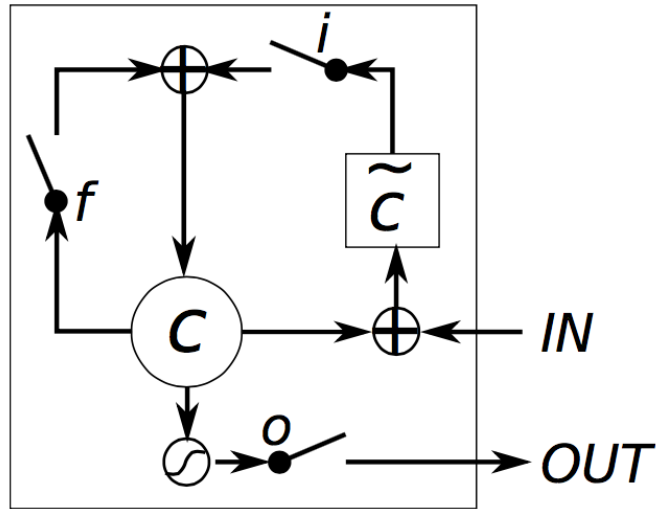
$$r_t = \sigma (W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh (W \cdot [r_t * h_{t-1}, x_t])$$

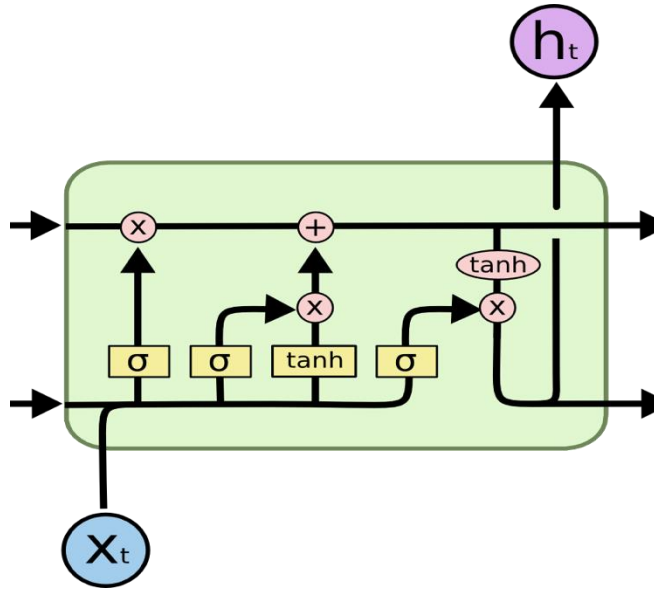
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Similar with LSTM but with only two gates and less parameters.
The “update gate” determines how much of previous memory to be kept.
The “reset gate” determines how to combine the new input with the previous memory.

Comparison of the gating mechanism



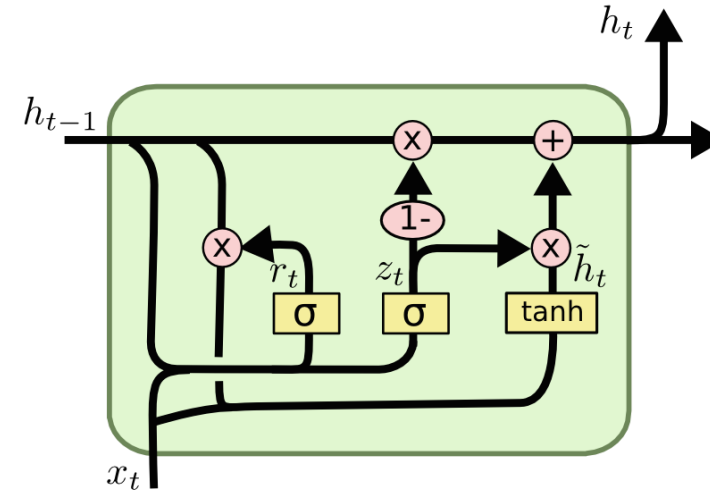
LSTM



$$\begin{aligned}
 f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\
 i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\
 \tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \\
 C_t &= f_t * C_{t-1} + i_t * \tilde{C}_t \\
 o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\
 h_t &= o_t * \tanh(C_t)
 \end{aligned}$$

VS.

GRU



$$\begin{aligned}
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\
 r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\
 \tilde{h}_t &= \tanh(W \cdot [r_t * h_{t-1}, x_t]) \\
 h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t
 \end{aligned}$$

Referencias

- <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- <https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>
- <https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>